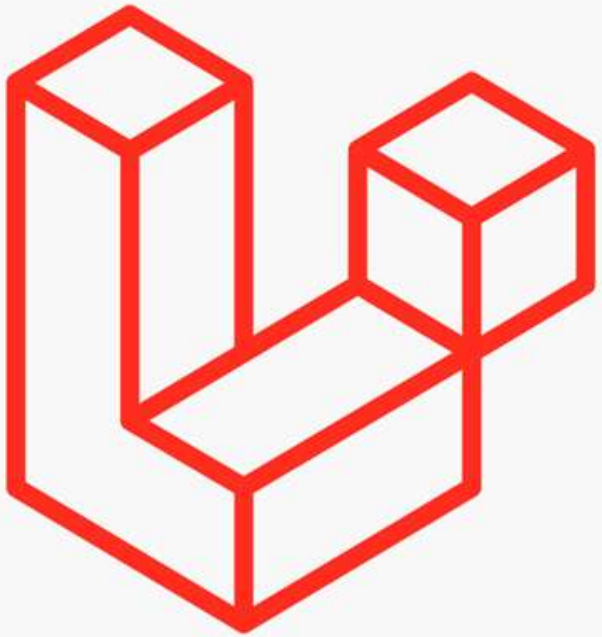


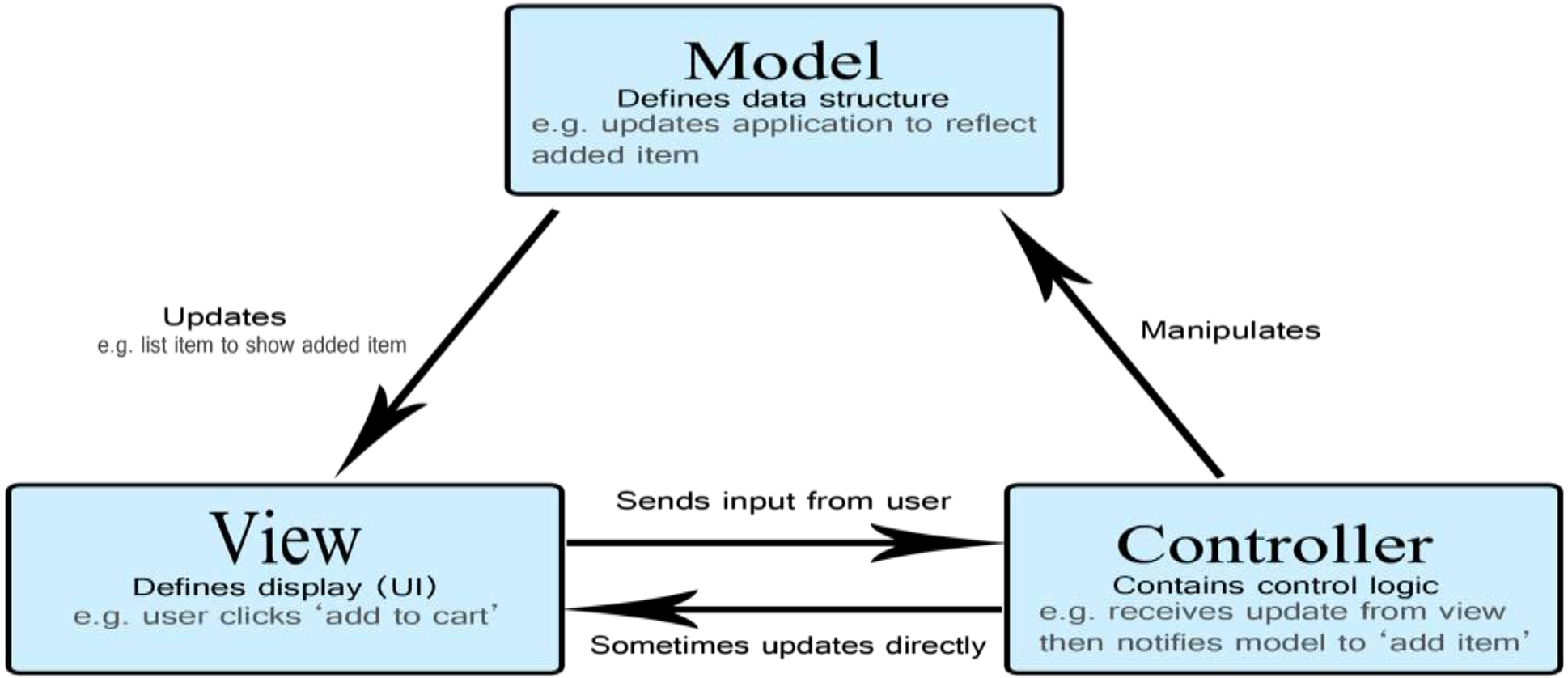
Laravel

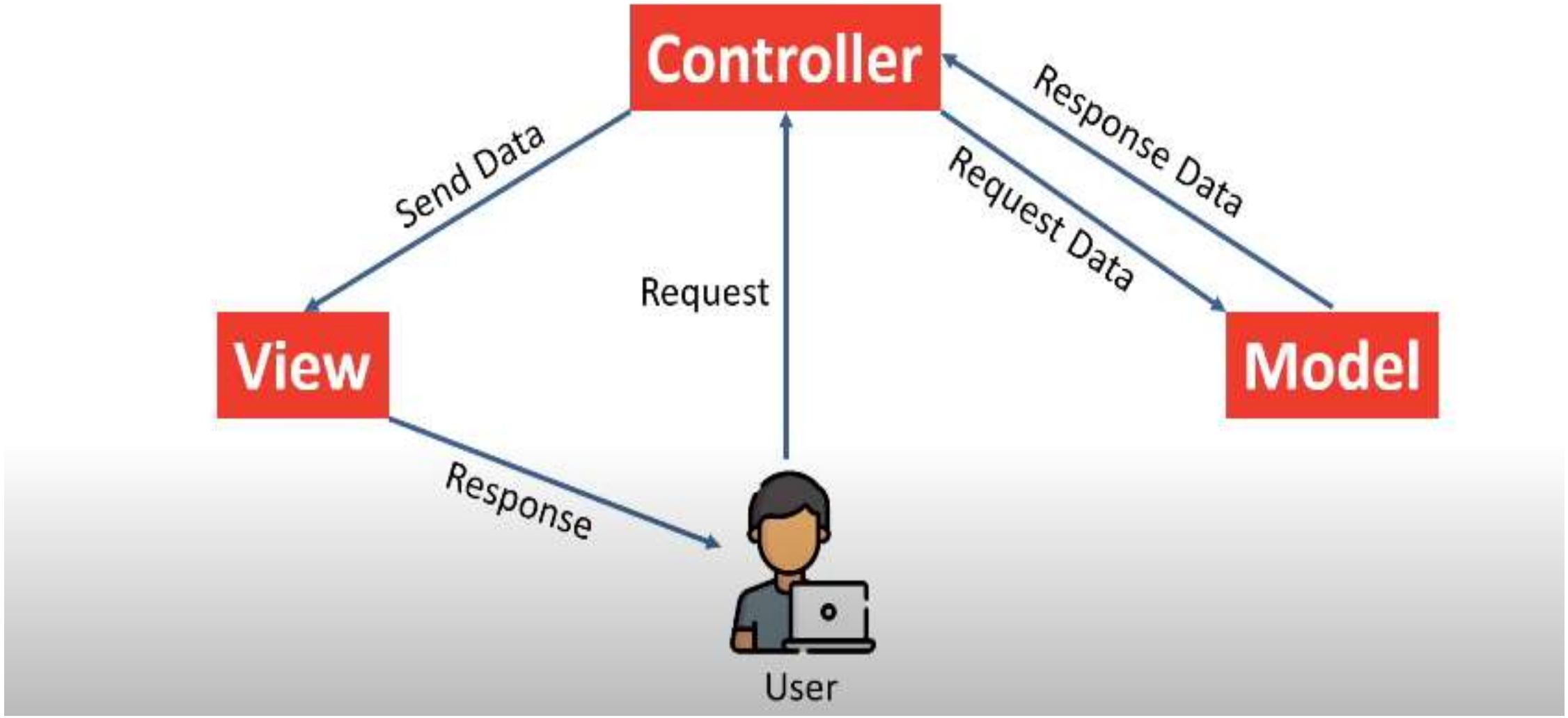
Framework?

A web framework is a software platform that provides a structured and standardized way to build web applications.

Popular Web Frameworks

What is MVC ?


Model
Defines data structure
e.g. updates application to reflect added item
Updates
e.g. list item to show added item
Manipulates
View
Defines display (UI)
e.g. user clicks 'add to cart'
Sends input from user
Sometimes updates directly
Controller
Contains control logic
e.g. receives update from view then notifies model to 'add item'


Controller
Send Data
Response Data
Request Data
Request
View
Model
Response
User

Benefits of MVC

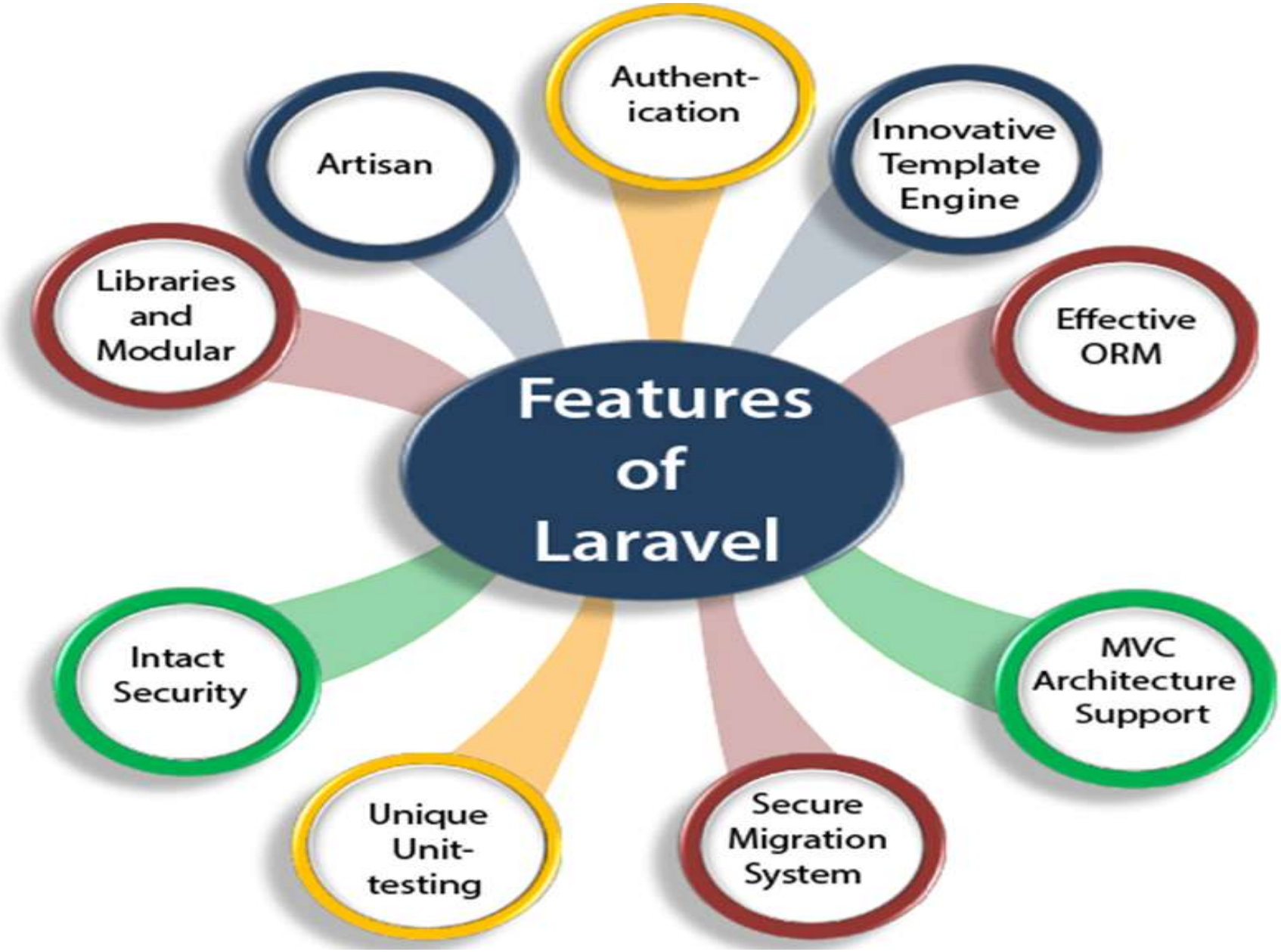
- Organized Code
- Independent Block
- Reduces the complexity of Web Applications
- Easy to maintain
- Easy to modify
- Code reusability
- Improved collaboration
- Platform independence

Laravel

Laravel is a PHP-based web framework for building high-end web applications using its significant and graceful syntaxes.

Taylor Otwell developed Laravel in July 2011, and it was released more than five years after the release of the Codeigniter framework.


LOVELY PROFESSIONAL UNIVERSITY
Laravel is the strongest contender in the PHP ecosystem simply because it includes the features needed to build modern web applications.
Taylor Otwell,
Creator of Laravel, Forge, Envoyer


Features of Laravel
Authent-ication
Innovative Template Engine
Effective ORM
MVC Architecture Support
Secure Migration System
Unique Unit-testing
Intact Security
Libraries and Modular
Artisan

Authentication

Authentication is the most important factor in a web application, and developers need to spend a lot of time writing the authentication code. Laravel contains an inbuilt authentication system, you only need to configure models, views, and controllers to make the application work.

Innovative Template Engine

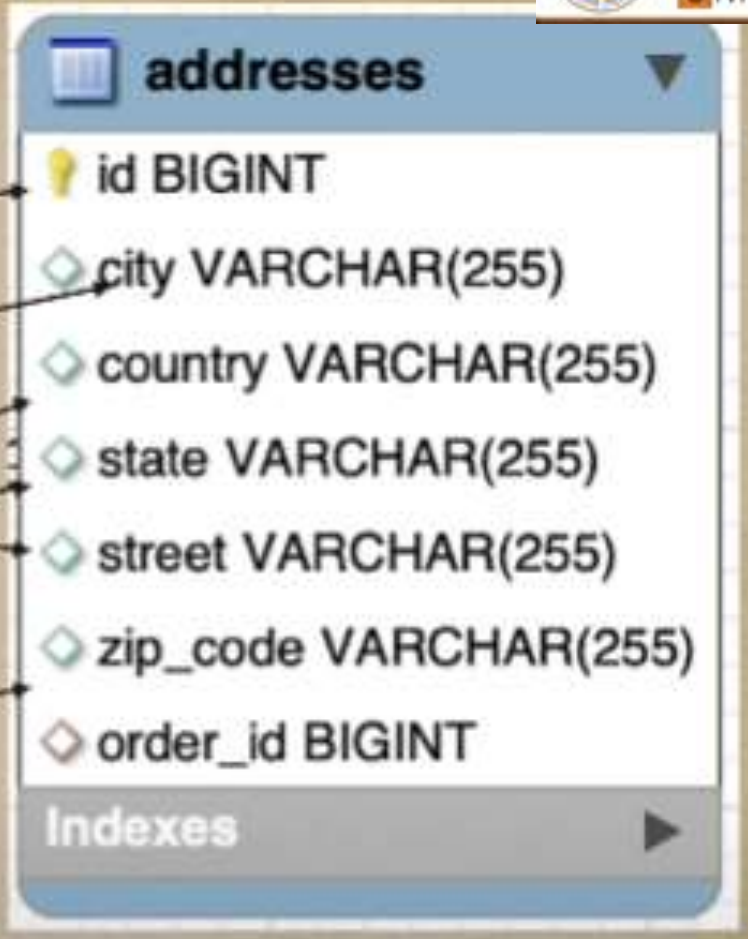
Laravel's innovative template engine, called **Blade**, is designed to simplify the process of creating and managing views (templates) in web applications. Blade provides a clean, expressive syntax that allows developers to write views with ease while providing powerful features for dynamic content rendering.

Effective Object Relational Mapping

When building applications, developers often use object-oriented programming languages like Python, Java, or php to represent entities and their relationships**. These entities are typically classes that have attributes and methods. However, databases primarily store data in tables with rows and columns**.

ORMs provide a solution to this problem by abstracting the database interactions and mapping the objects to database tables, thereby enabling developers to work with their preferred object-oriented paradigm without worrying about the underlying database structure.

```java
@Entity
@Table(name="addresses")
public class Address {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="id")
    private Long id;

    @Column(name="street")
    private String street;

    @Column(name="city")
    private String city;

    @Column(name="state")
    private String state;

    @Column(name="country")
    private String country;

    @Column(name="zip_code")
    private String zipCode;
```




```java
Address address = new Address();
address.setCity("Pune");
address.setStreet("Kothrud");
address.setState("Maharashtra");
address.setCountry("India");
address.setZipCode("411047");
```

Result Grid Filter Rows: Search

id	city	country	state	street	zip_code
2	Pune	India	Maharashtra	Kothrud	411047
3	Pune	India	Maharashtra	Kothrud	411047

Laravel's support for the Model-View-Controller (MVC) architectural pattern brings several benefits to web application development. MVC is a software design pattern that **separates the application logic into three interconnected components: Model, View, and Controller**.

Secure Migration System

Laravel's migration system **simplifies the process of managing database changes in a structured and consistent manner**. It promotes version control of the database schema and helps developers work collaboratively, ensuring the security and integrity of the application's database.

Unique Unit Testing

Laravel seamlessly integrates with **PHPUnit**, one of the most widely used and well-established testing frameworks in the PHP ecosystem. PHPUnit offers a rich set of assertion methods and testing features, making it easier to write clear and concise tests.

Laravel Dusk, the built-in browser testing tool, provides an excellent way to perform end-to-end tests and **interact with your application as a real user would**.

Intact Security

Laravel uses **bcrypt hashing** to securely store user passwords. Bcrypt is a strong and slow hashing algorithm, making it difficult for attackers to crack passwords using brute-force attacks.

Libraries and Modules

Laravel is very popular, as **some object-oriented libraries and pre-installed libraries are added in this framework; these pre-installed libraries are not added in other PHP frameworks**. One of the most popular libraries is an **authentication library that contains some useful features such as password reset, monitoring active users, bcrypt hashing, and CSRF protection**. This framework is divided into several modules that follow the PHP principles, allowing the developers to build responsive and modular apps.

Artisan

The Laravel framework provides a built-in tool for the command line known as Artisan that **performs the repetitive programming tasks** that do not allow the PHP developers to perform manually. These artisans can also be **used to create the skeleton code, database structure, and its migration**, so it makes it easy to manage the database of the system. It also **generates the MVC files through the command line. Artisan also allows the developers to create their own commands**.

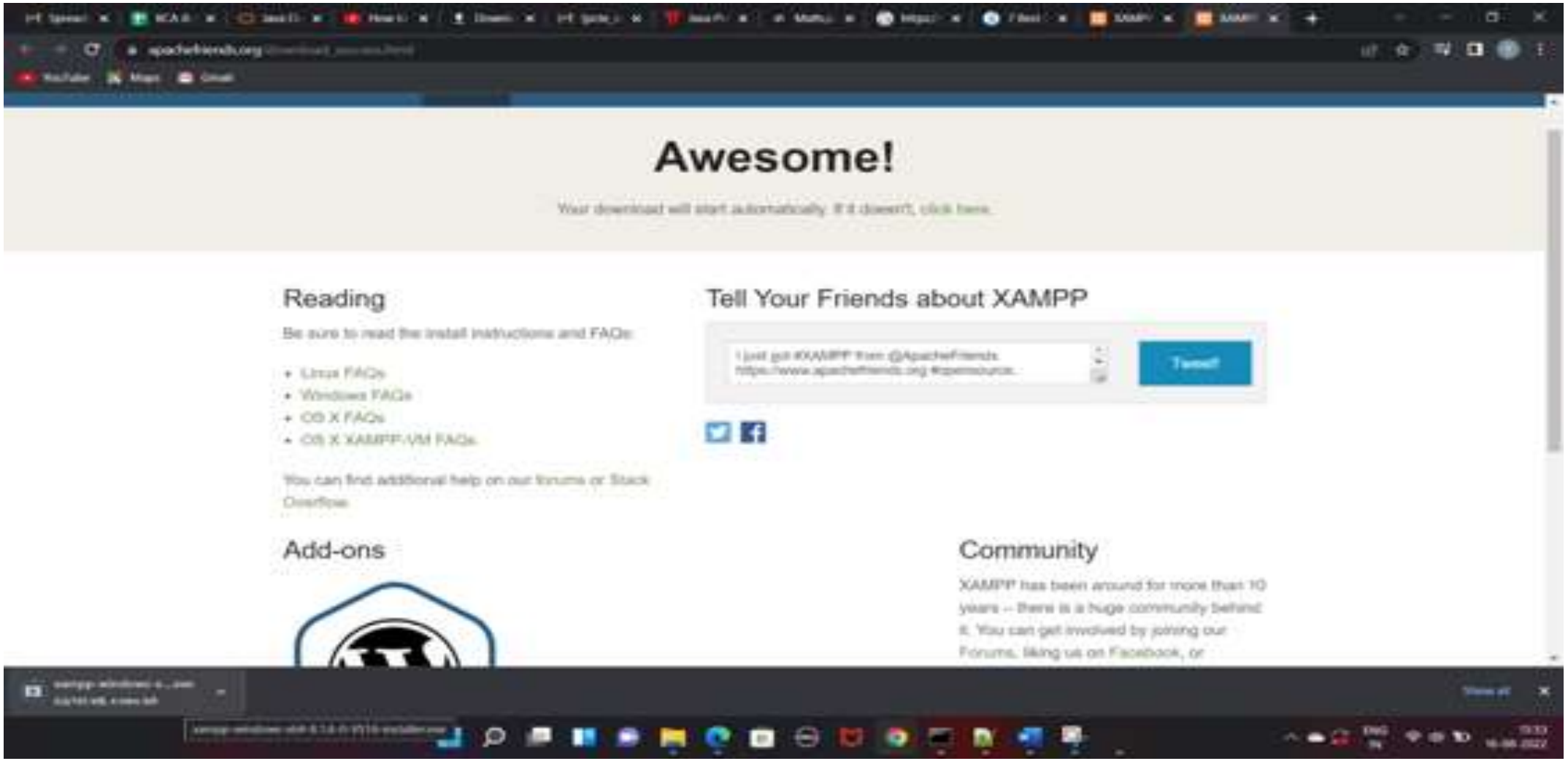
Requirements for Laravel project

- Xampp (supporting php> 8.2)
- Composer
- VS-Code

GO to apachefriends.org

Go To XAMPP for Windows

XAMPP will start downloading

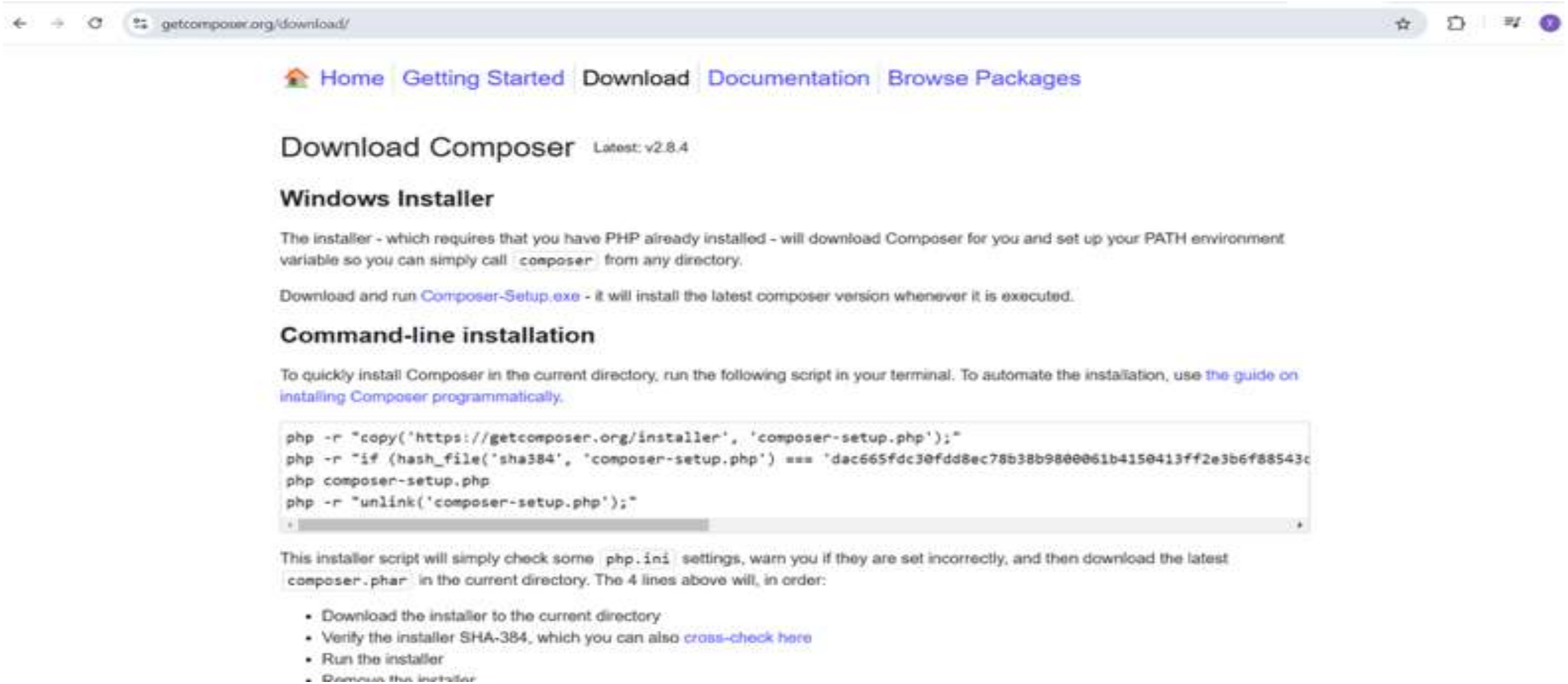


Composer

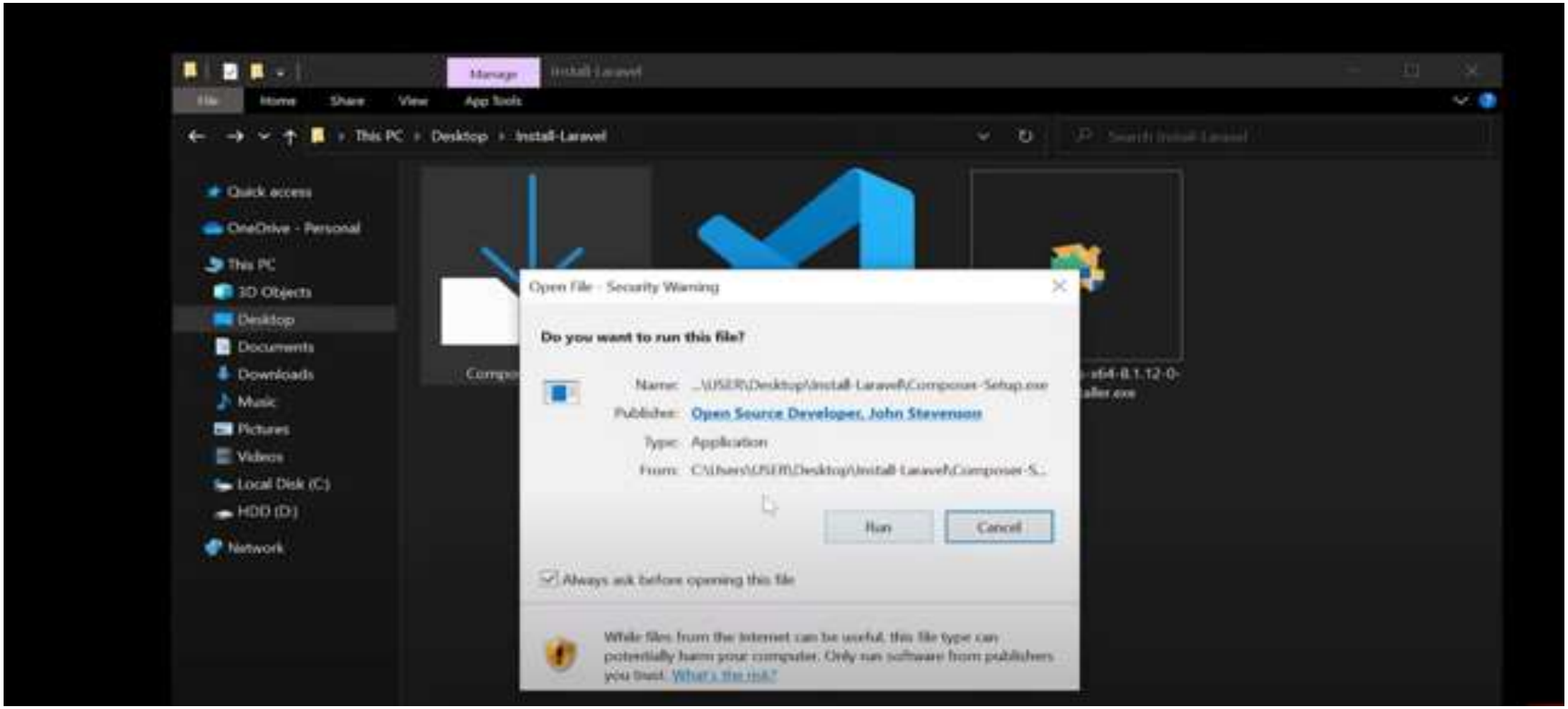
- For managing the project's dependencies.
- Its main purpose is to simplify the process of including external libraries and packages into your Laravel application.

Go to getcomposer.org

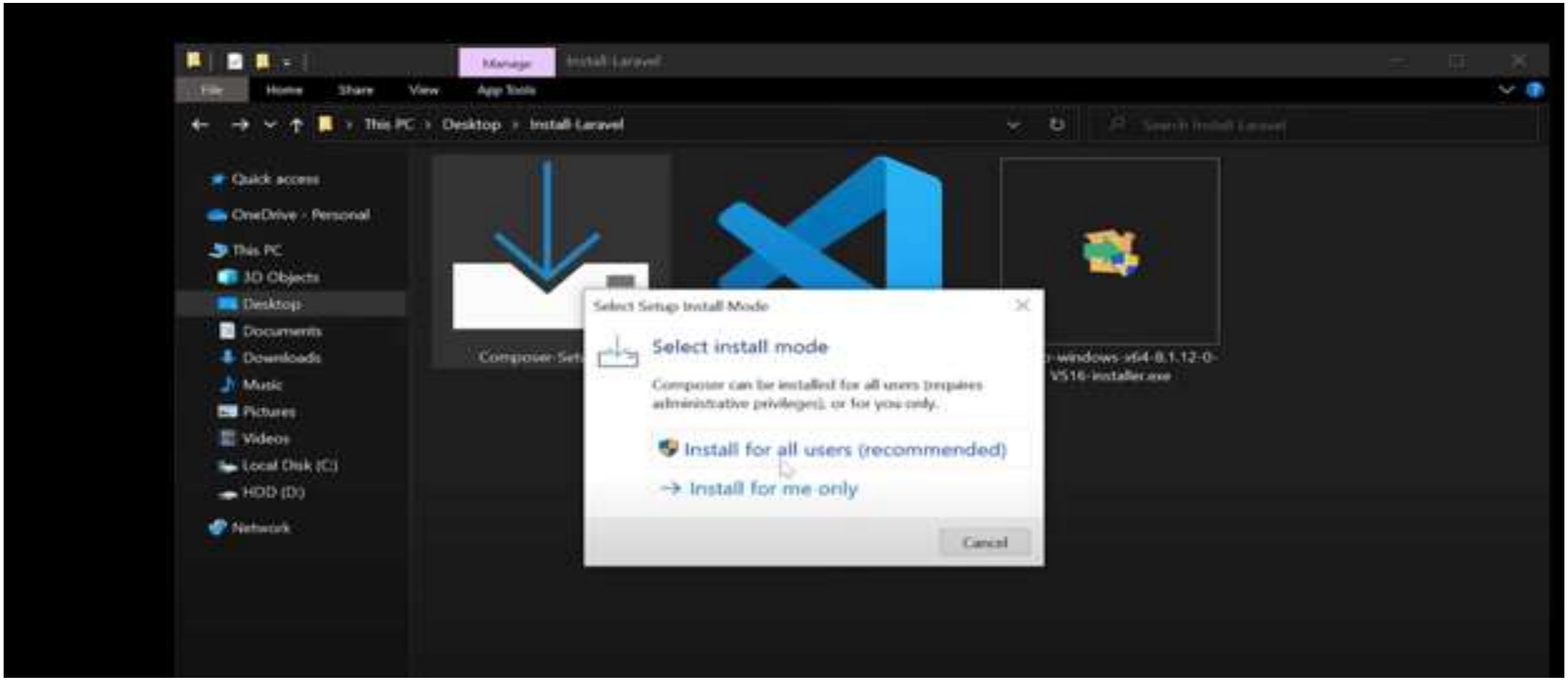
After clicking on Downloads run Composer-setup.exe file



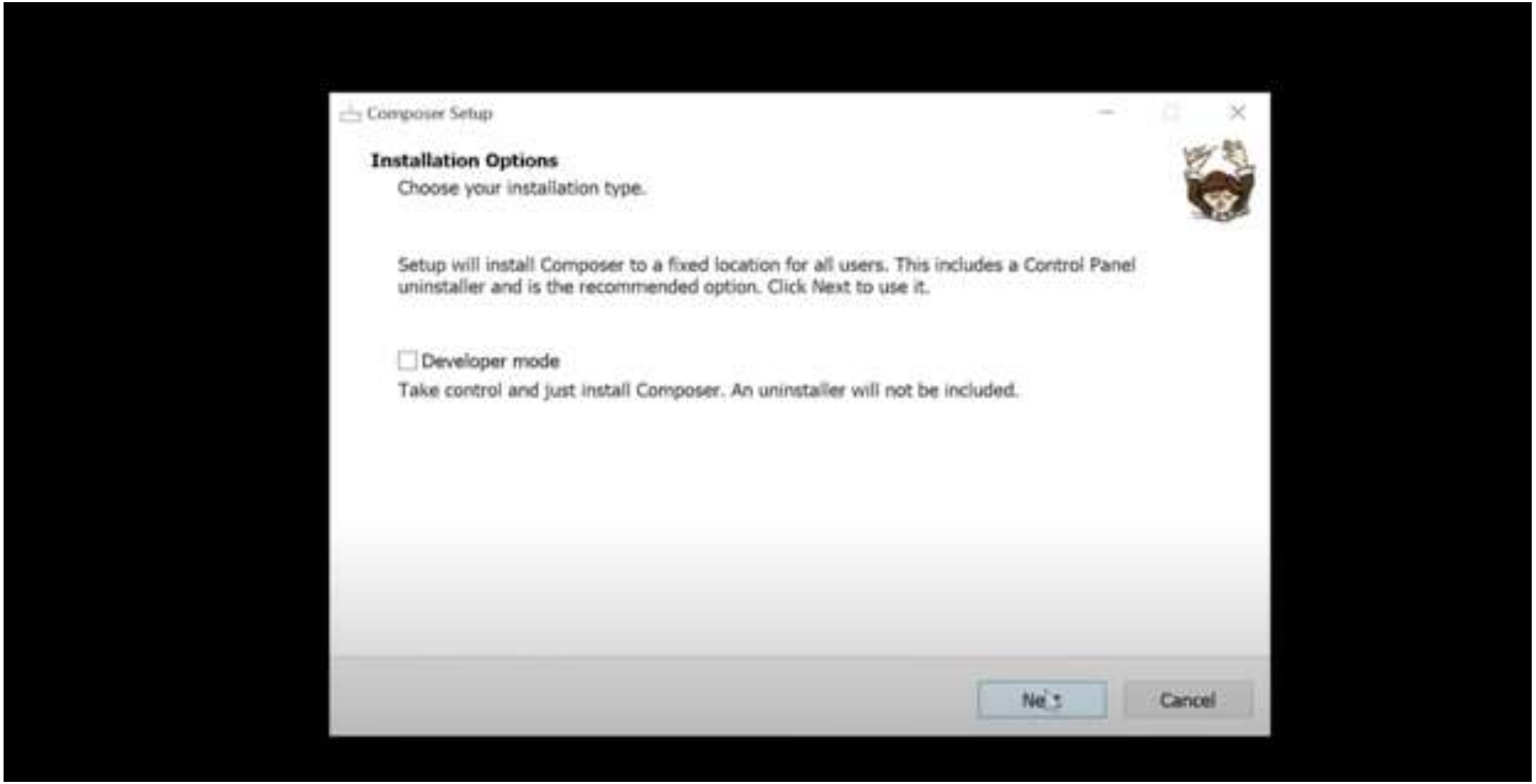
Click on Run


Manage
Install-Laravel
File
Home
Share
View
App Tools
This PC
Desktop
Install-Laravel
Quick access
OneDrive - Personal
This PC
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
Local Disk (C:)
HDD (D:)
Network
Open File - Security Warning
Do you want to run this file?
Name:
Publisher:
Open Source Developer, John Stevenson
Type:
Application
From:
Run
Cancel
Always ask before opening this file
While files from the Internet can be useful, this file type can potentially harm your computer. Only run software from publishers you trust. What's the risk?

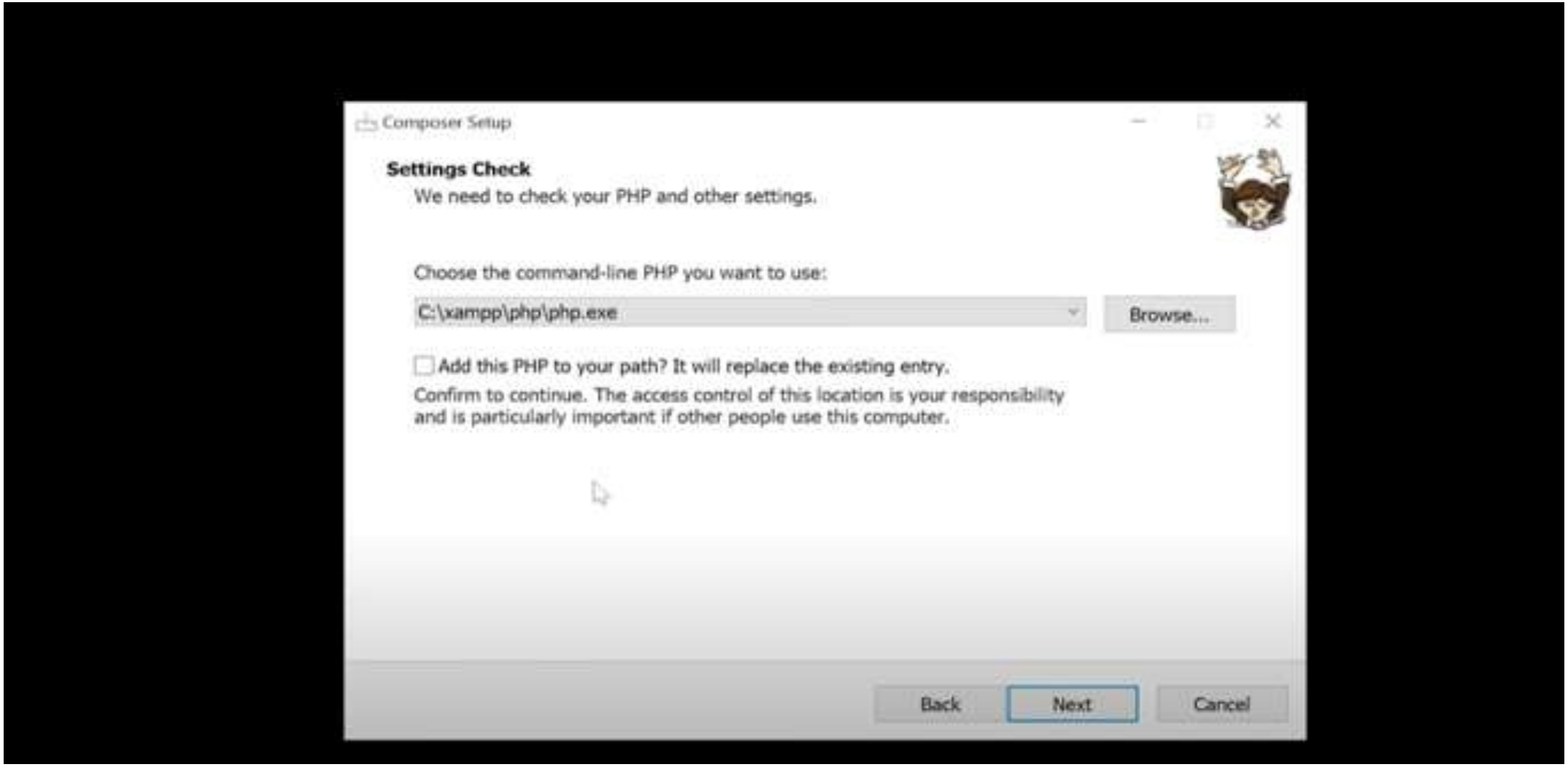
Click on Install for all users



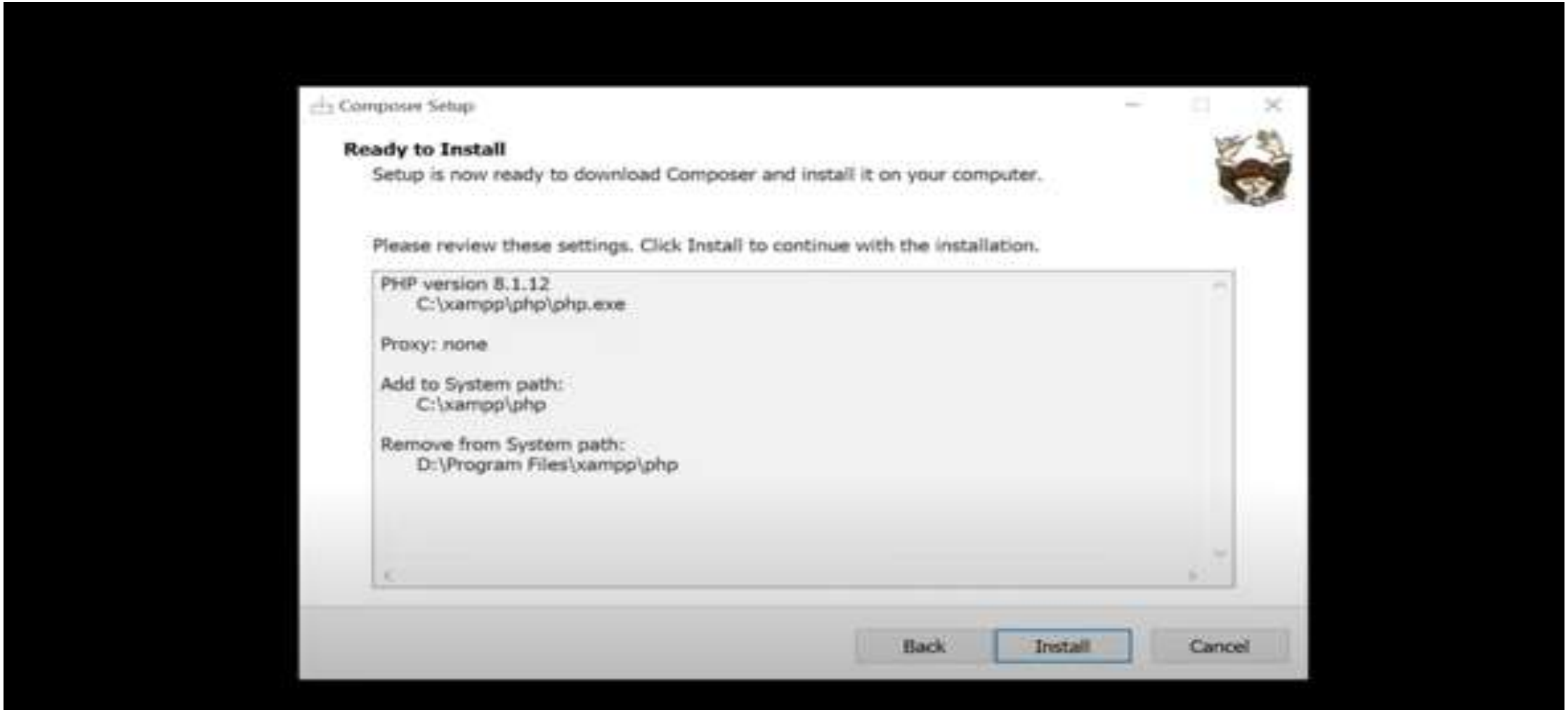
Click on Next



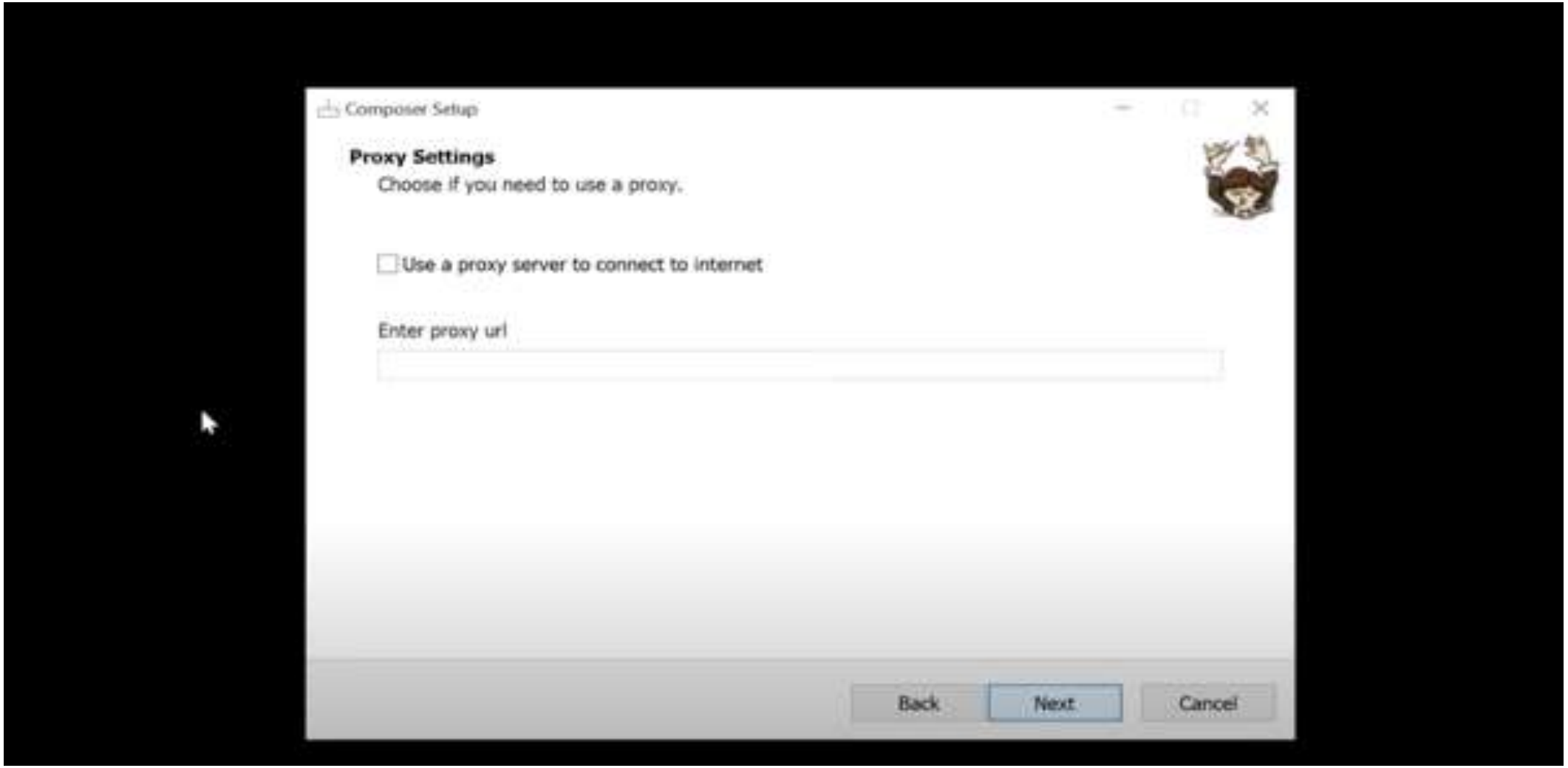
Click on Check Box and click on Next



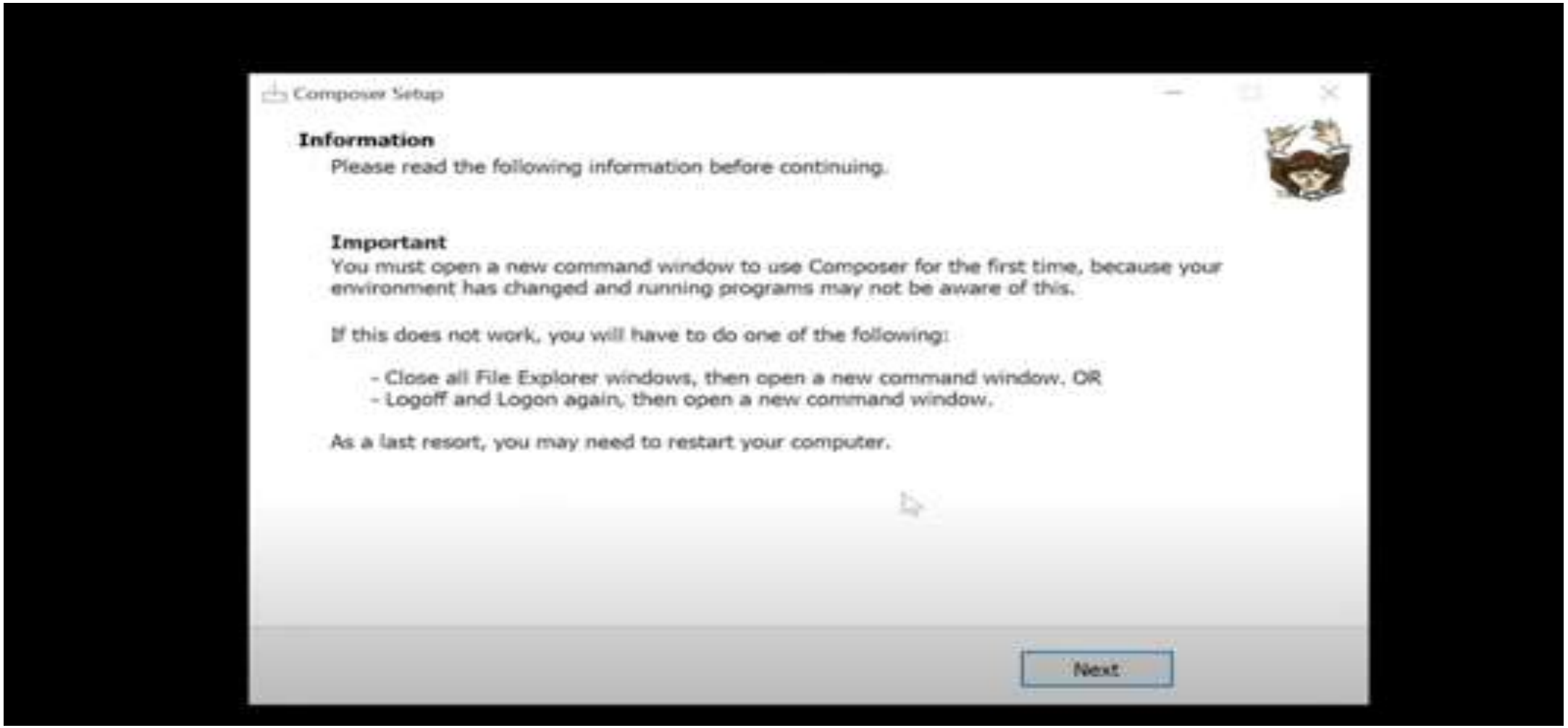
Click on Install



Click on Next



Click Next


Composer Setup
Information
Please read the following information before continuing.
Important
You must open a new command window to use Composer for the first time, because your environment has changed and running programs may not be aware of this.
If this does not work, you will have to do one of the following:
- Close all File Explorer windows, then open a new command window. OR
- Logoff and Logon again, then open a new command window.
As a last resort, you may need to restart your computer.
Next

Check PHP Version

```
php -v
```

Check Git Version

```
git --version
```

Check Composer Version

```
composer --version
```

Or

```
composer
```

Check Laravel version

```
laravel -v
```

Installation of Laravel

- Global Installation

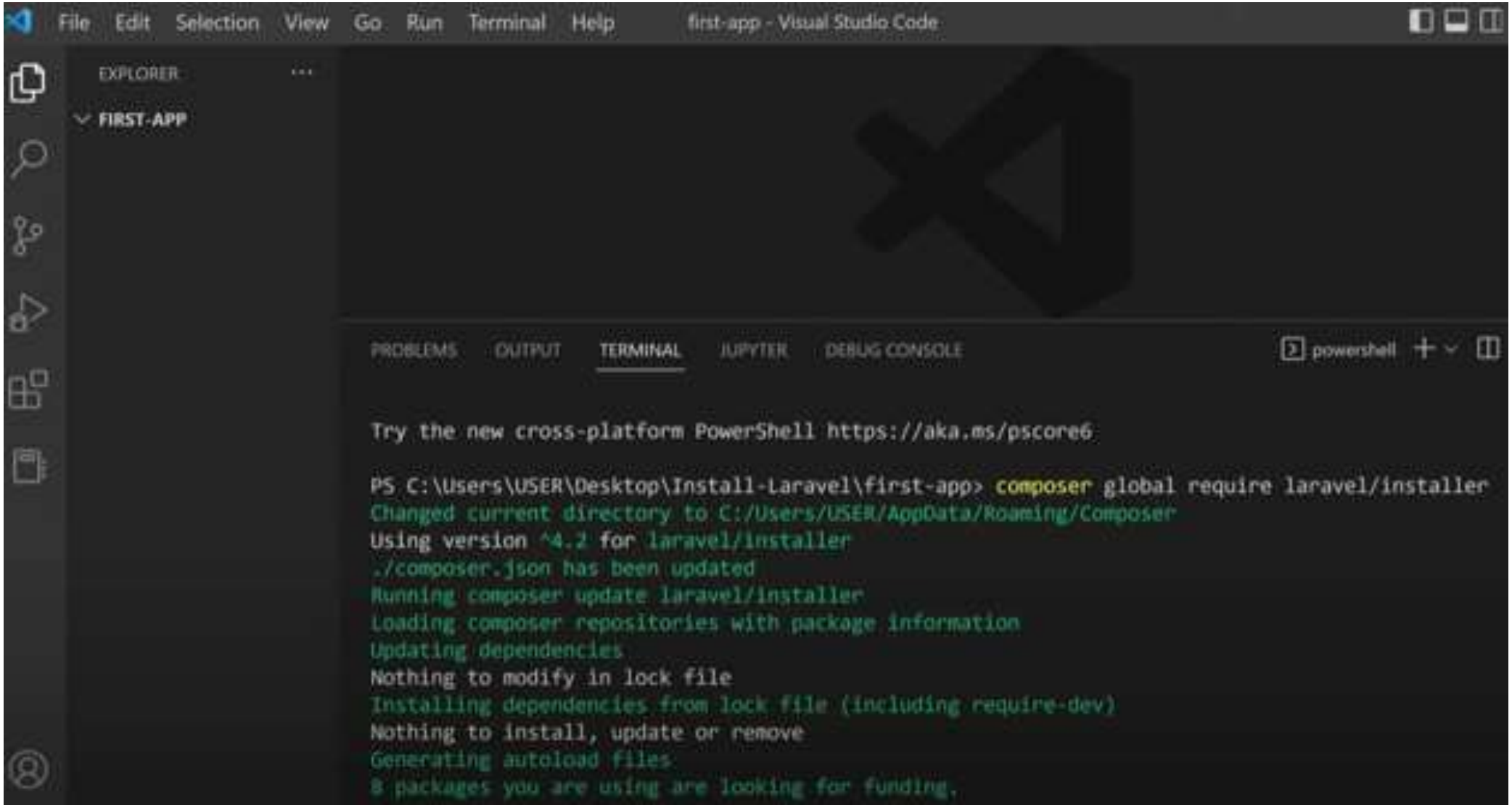
  1- **composer global require laravel/installer** (one-time command)

  2- **laravel new first-app** (for new app)

- Per-project Installation

  composer create-project laravel/laravel first-app (every time for new app)

Open the folder in VS Code and run the command (Terminal New Terminal)



Issues and solutions- When running the composer global require laravel/installer

- Git installation,
- modifying the php.ini file,
- or environment configuration.

Problem:

- Composer requires Git to download dependencies (like laravel/installer) from repositories. If Git is not installed or not properly set up, you'll see errors like:bashCopy code
- Failed to clone ... repository not found
- Need XAMPP Latest Version
- composer clear –cache on terminal
- Then install composer

Solution:

- Verify Git Installation:
  - Run git --version in the terminal to check if Git is installed.
  - Install Git:On Windows: Download Git from git-scm.com and follow the installation instructions.

- Modifying php.ini File:
  - The php.ini file often needs to be updated, which involves removing the semicolon (;) in front of specific lines to enable extensions.

extension=zip

Remove ; from above line

Running a Laravel app

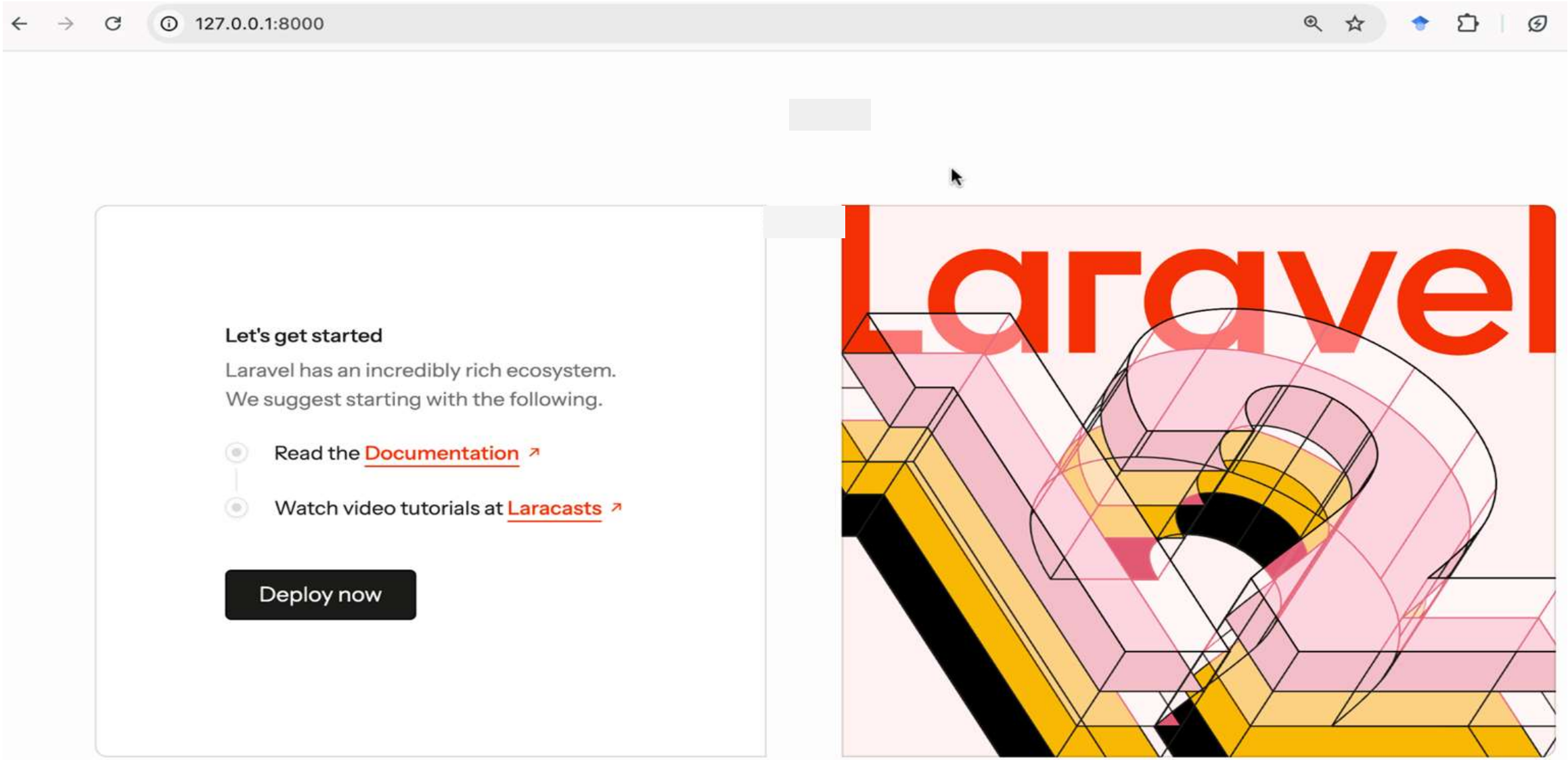
cd first-app

php artisan serve

```
PS D:\Laravel Projects> cd first
PS D:\Laravel Projects\first> php artisan serve
  forking is not supported on this platform

  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server

PS D:\Laravel Projects\first> php artisan make view::data
Laravel Framework 11.37.0
```

- The artisan serve command in Laravel is a way to start a built-in PHP development server for your Laravel project.
- This command is commonly used during local development to quickly serve your application without needing to set up a full web server like Apache.
- Starts a local development server at http://127.0.0.1:8000 by default.
- It allows developers to test their Laravel application locally in a browser.
- Default Port Number: 8000

Laravel Home Page



Artisan

- Command line interface used in Laravel is called Artisan.
- It includes a set of commands which assists in building a web application.
- Artisan is Laravel's **command-line interface (CLI)** that provides powerful commands for automating tasks like database migrations, model generation, route listing, and caching.
- It helps developers save time by reducing manual work.

- php artisan list
  a. Displaying All Artisan Commands

A. Application Management

Command	Description
`php artisan serve`	Starts a development server at `http://127.0.0.1:8000/`
`php artisan down`	Puts the application into maintenance mode
`php artisan up`	Brings the application back online
`php artisan about`	Displays Laravel version & system info

B. Route Management

Command	Description
`php artisan route:list`	Displays all registered routes
`php artisan route:cache`	Caches routes for faster performance
`php artisan route:clear`	Clears the cached routes

C. Model & Database Commands

Command	Description
`php artisan make:model Product`	Creates a new model (`app/Models/Product.php`)
`php artisan make:model Product -m`	Creates a model with a migration file
`php artisan make:migration create_products_table`	Generates a migration file
`php artisan migrate`	Runs all pending migrations
`php artisan migrate:rollback`	Rolls back the last migration
`php artisan db:seed`	Runs database seeders
`php artisan make:seeder ProductSeeder`	Creates a new database seeder

D. Authentication & Authorization

Command	Description
`php artisan make:auth`	Generates authentication scaffolding (for Laravel 5.x, 6.x)
`php artisan make:policy PostPolicy`	Creates a new policy for authorization

E. Caching & Performance Optimization

Command	Description
`php artisan cache:clear`	Clears the application cache
`php artisan config:cache`	Caches configuration files
`php artisan view:cache`	Caches Blade templates
`php artisan view:clear`	Clears the cached views

F. Queues & Jobs

Command	Description
`php artisan queue:work`	Starts processing queued jobs
`php artisan queue:listen`	Listens for new jobs in the queue
`php artisan queue:failed`	Displays failed jobs
`php artisan queue:retry all`	Retries all failed jobs

G. Event & Listener Commands

Command	Description
`php artisan event:list`	Lists all registered events
`php artisan make:event OrderShipped`	Creates a new event class
`php artisan make:listener SendOrderNotification`	Creates an event listener

What does MVC stand for in software development?

a) **Model-View-Controller**

b) Most Valuable Code

c) Managed View Components

d) Modular-Visual-Creation

Always Remember……

MVC is indeed a **broader design pattern** that defines the high-level architecture of an application, emphasizing the separation of concerns. However, when implementing the MVC pattern**, the actual code is organized into separate layers to achieve modularity, maintainability, and scalability**.

What is the primary purpose of the Model in the MVC architecture?

a) Handles user interactions and presentation logic

b) **Represents the data and business logic of the application**

c) Renders the user interface and displays data

d) Manages the communication between the Model and View

What is Laravel?

a) A programming language

b) A JavaScript library

c) A PHP web application framework

d) A database management system

Do you know?

Originally, PHP stood for "**Personal Home Page**," as it was created to manage Rasmus Lerdorf's personal website in 1994.

Which command is used to create a new Laravel project using Composer?

a) **composer create-project Laravel app-name**

b) composer new laravel/laravel

c) **laravel new app-name**

d) composer new-project laravel/laravel

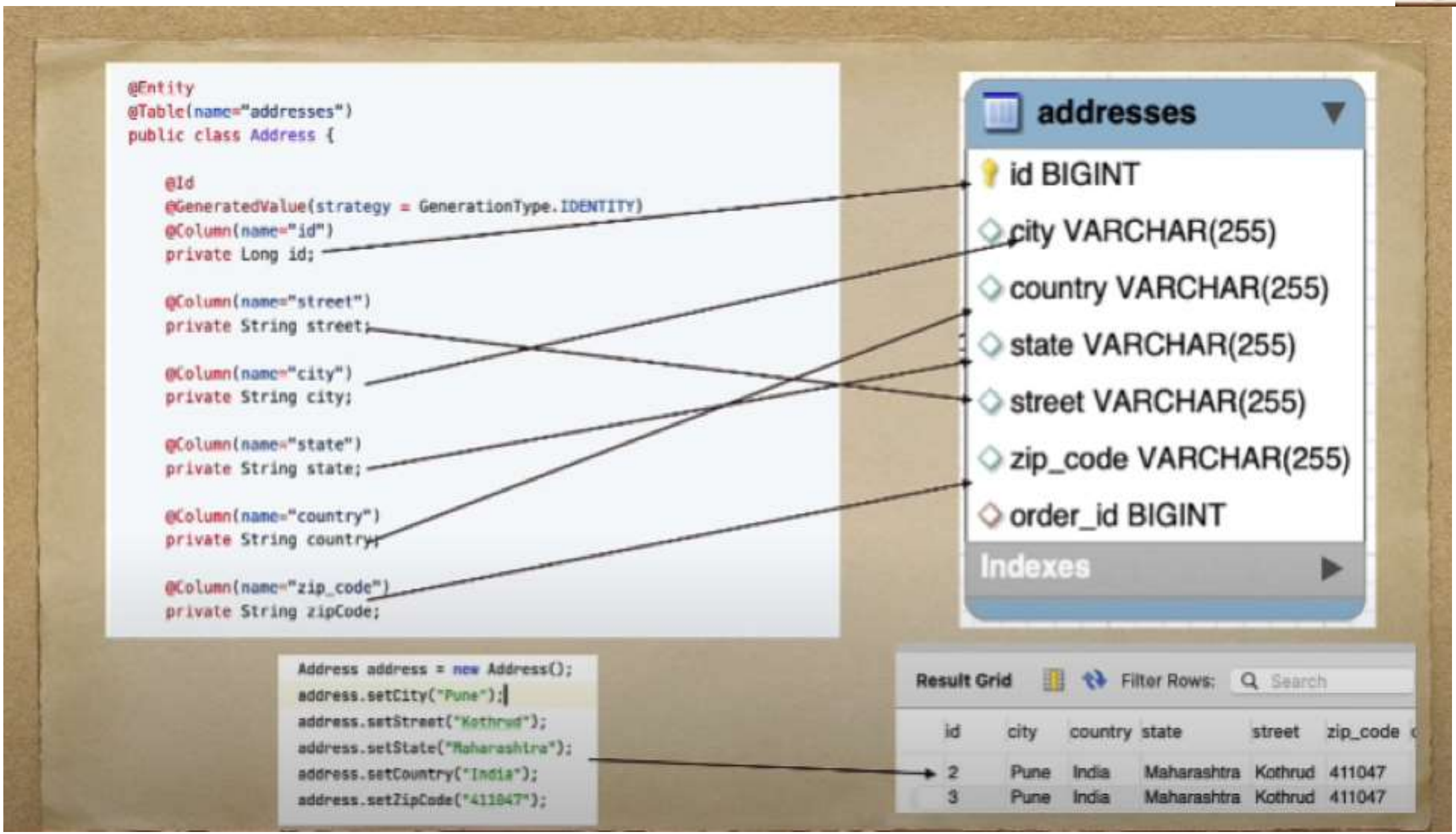
What is Eloquent in Laravel?

a) A templating engine

b) A database management system

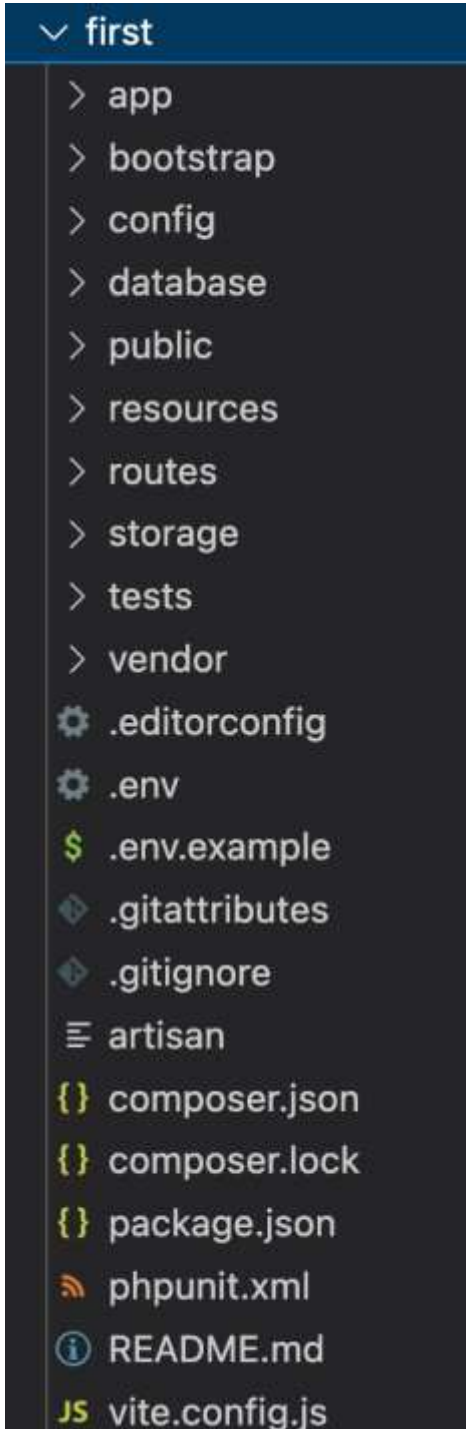
c) A PHP extension for performance optimization

d) **An ORM (Object-Relational Mapping) for database interactions**


@Entity
@Table(name="addresses")
public class Address {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name="id")
private Long id;
@Column(name="street")
private String street;
@Column(name="city")
private String city;
@Column(name="state")
private String state;
@Column(name="country")
private String country;
@Column(name="zip_code")
private String zipCode;
addresses
id BIGINT
city VARCHAR(255)
country VARCHAR(255)
state VARCHAR(255)
street VARCHAR(255)
zip_code VARCHAR(255)
order_id BIGINT
Indexes
Address address = new Address();
address.setCity("Pune");
address.setStreet("Kothrud");
address.setState("Maharashtra");
address.setCountry("India");
address.setZipCode("411047");
Result Grid
Filter Rows:
Search
id city country state street zip_code
2 Pune India Maharashtra Kothrud 411047
3 Pune India Maharashtra Kothrud 411047

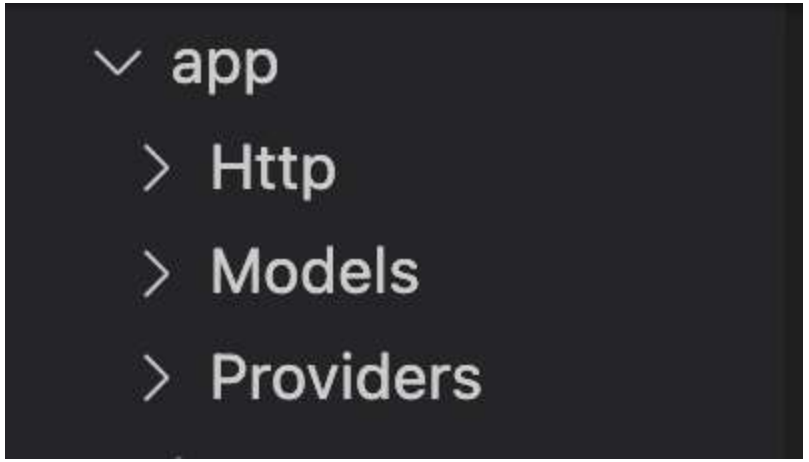
Directory Structure

The default Laravel application structure is intended to provide a great starting point for both large and small applications. **But you are free to organize your application however you like**.



App -> Http

This directory houses the controllers, middleware, and form requests. **Controllers handle HTTP requests and responses, middleware provides a way to filter incoming HTTP requests, and form requests handle form validation logic**.



App -> Models

In this directory, you define the **Eloquent ORM models**. Eloquent is Laravel's built-in ORM that allows you to interact with the database using an expressive syntax.

Providers

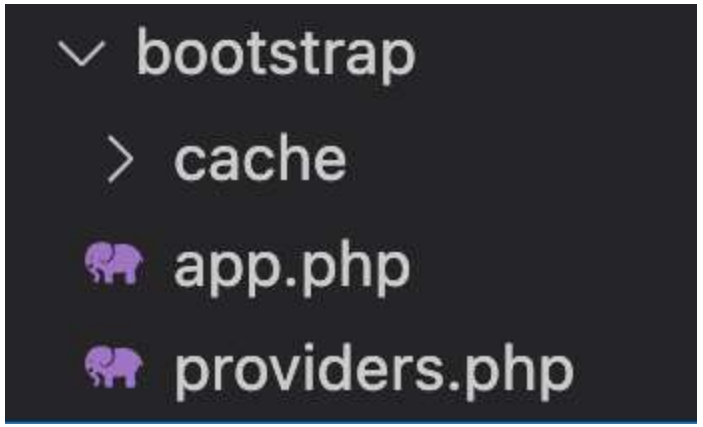
- Stores service provider classes.
- Service providers are responsible for bootstrapping your application.
- By default, Laravel includes several providers, such as AppServiceProvider, AuthServiceProvider, and EventServiceProvider.

Do you know ?

Laravel drew inspiration from several other frameworks, including **Ruby on Rails, ASP.NET MVC, and Sinatra**. It combines the best features from these frameworks and adds its own unique elements.

The Bootstrap Directory

The bootstrap directory contains files responsible for bootstrapping the Laravel application.



- app.php: This file loads the basic application configuration and **creates an instance of the Laravel application**.
- cache: The cache directory contains compiled service container and configuration files. These files improve the **application's performance by reducing the bootstrapping time**.

In a standard Laravel project, which directory contains the main application logic and business-specific code?

a) resources

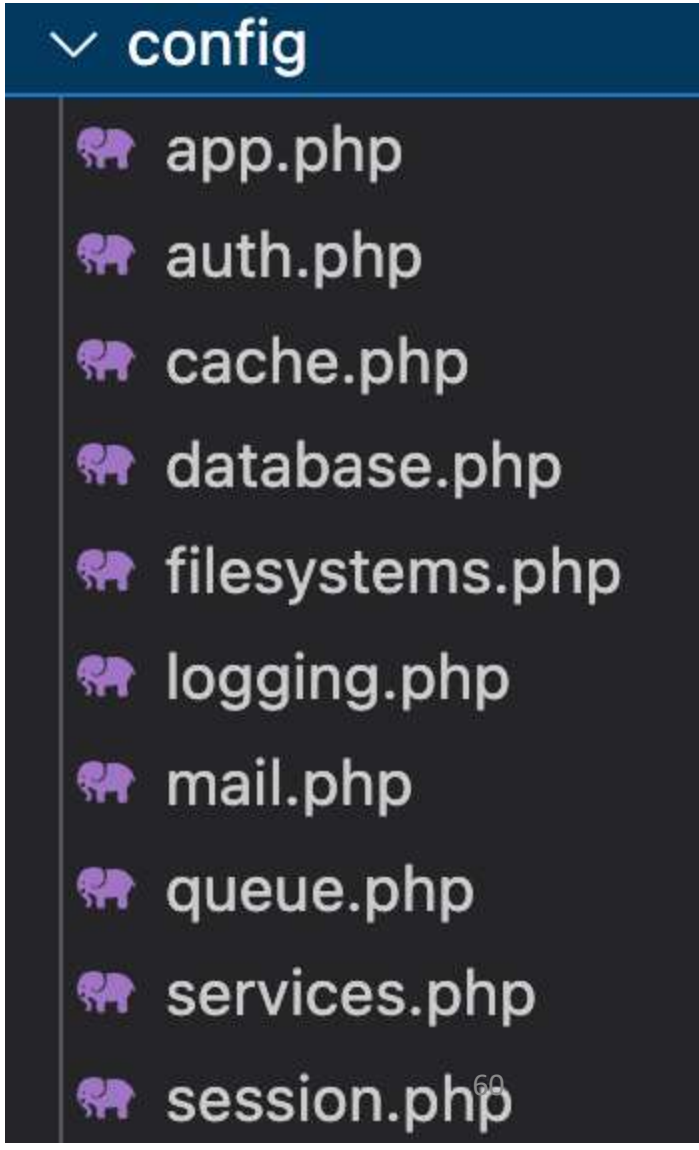
b) public

c) app

d) database

Config Directory

The config directory houses various **configuration files for different components of the Laravel application**. You can customize settings like database connections, application settings, cache configurations, and more from these files.



Database Directory

The database directory contains **database-related files**, including migrations, seeds, and factories.

Database -> Factories

Factory classes are used to define data blueprints that can be **used to create dummy data for testing**.

Database -> Migration

Migrations are **version control for your database**. They allow you to modify the database schema and **keep it in sync with your application code**.

Database -> Seeds

Seed classes **allow you to populate your database with sample data for testing** or initial setup.

In Laravel, where do you define the database schema and migrations for creating and modifying database tables?

a) resources/views

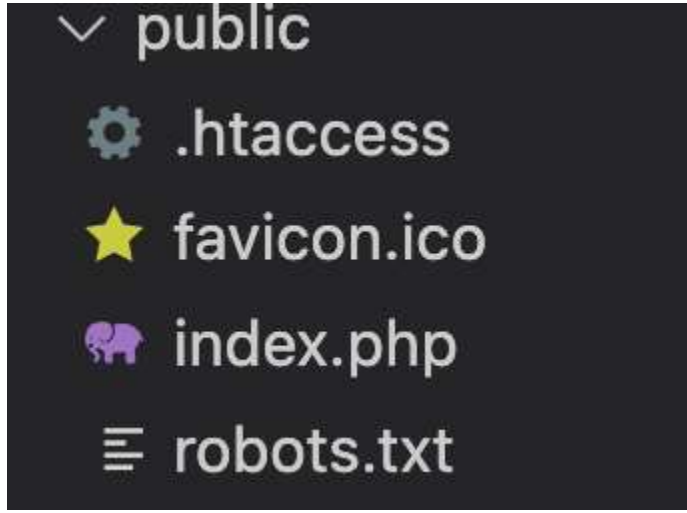
b) app

c) database/migrations

d) config

Public Directory

- The public directory is the web server's document root. It **contains the index.php file, which is the entry point for all HTTP requests**.
- configures autoloading
- It is the directory where all public-facing files reside, such as assets, images, Javascipt, CSS and favicon icon and the primary PHP file that handles incoming requests.



Which directory is used for storing publicly accessible assets such as images, stylesheets, and JavaScript files?

a) resources

b) **public**

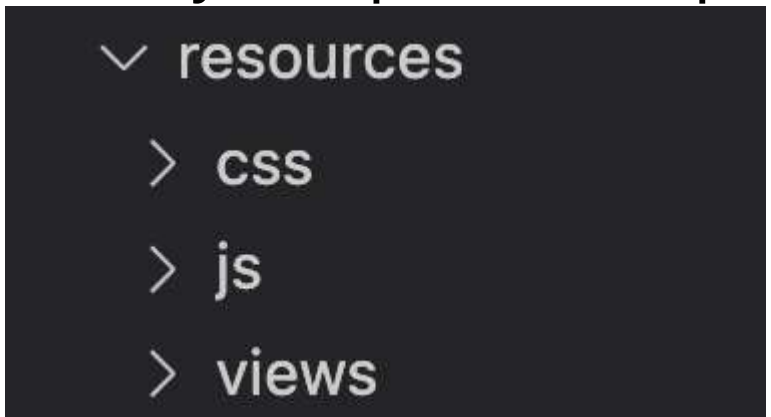
c) storage

d) app

Resource Directory

The resources directory contains the views, language files, and assets (CSS, JavaScript, and images) for the application.

- **Views**: The views directory **stores Blade templates** that define the UI of the application.
- **Lang**: The lang directory **holds language files for localization** and internationalization purposes.
- **Assets**: The assets directory **contains raw CSS, JavaScript, and other assets**. These assets are usually compiled and optimized using Laravel Mix.



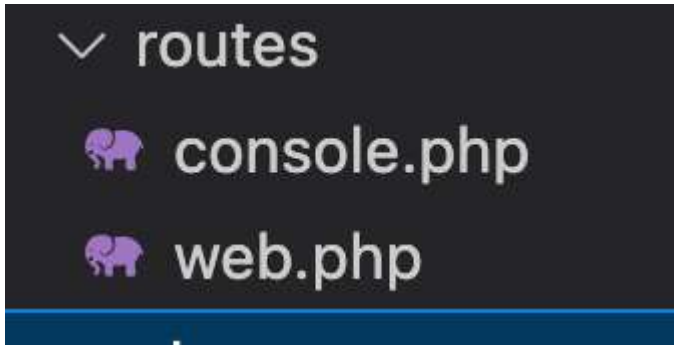
Do you Know?

It takes less than a second (**0.9 seconds** to be precise) for users to form their opinions on their initial visit to a web page.

Routes Directory

The routes directory contains route definition files.

- **web.php**: This file contains routes that are typically associated with web-based user interfaces.
- **api.php**: This file contains routes for API endpoints, usually meant to return JSON responses.



routes/web.php (opened at which link)

- In Laravel, the routes/web.php file is where you define routes for web interfaces of your application.
- These routes typically respond to requests made via a web browser and are grouped within the web middleware group, which provides features such as session handling, CSRF protection, and cookie encryption.
- Basic Route Example:

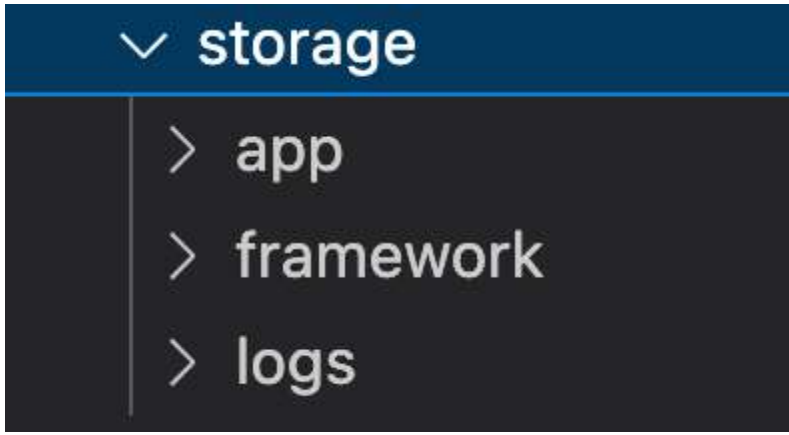
```
Route::get('/', function () {
    return view('welcome');
});
```

routes/console.php file (Console commands)

- The routes/console.php file in Laravel is where you define all custom Artisan commands for your application.
- This file allows you to extend and customize the functionality of Laravel's command-line interface (CLI)/terminals.
- Not used directly

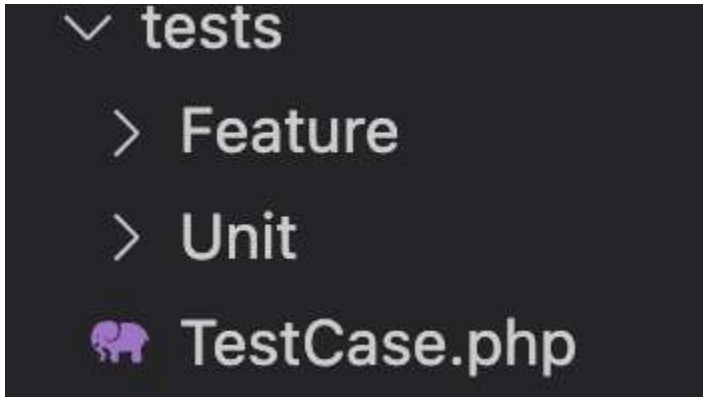
Storage Directory

The storage directory **holds various files generated by the application**, such as logs, cache, sessions, and uploaded files.



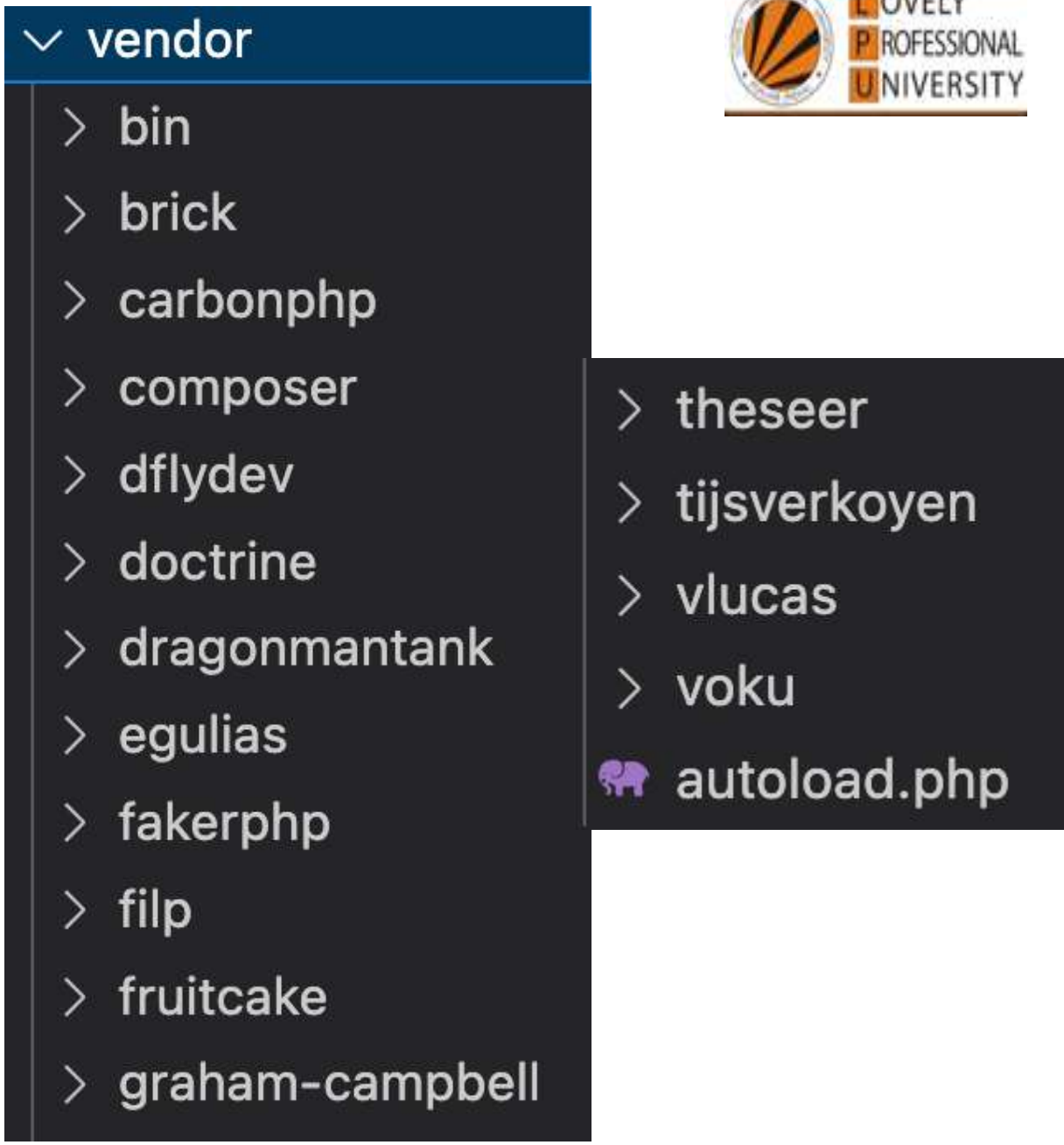
Test Directory

The tests directory **contains test cases** for the application.



Vendor Directory

The vendor directory **houses all the third-party dependencies installed via Composer**, including Laravel itself.



Vendor folder (mentioned in composer.json and stored in Vendor)

- The vendor folder in a Laravel project is where Composer stores all the **third-party dependencies required for the application.**
- It is automatically generated and managed by Composer and includes **libraries, frameworks, and tools specified in the composer.json file.**

Autoloading:

- The vendor/autoload.php file is the main entry point for Composer's autoloading mechanism.
- Laravel's bootstrap/app.php includes this file to autoload all necessary classes

Organizational Structure:

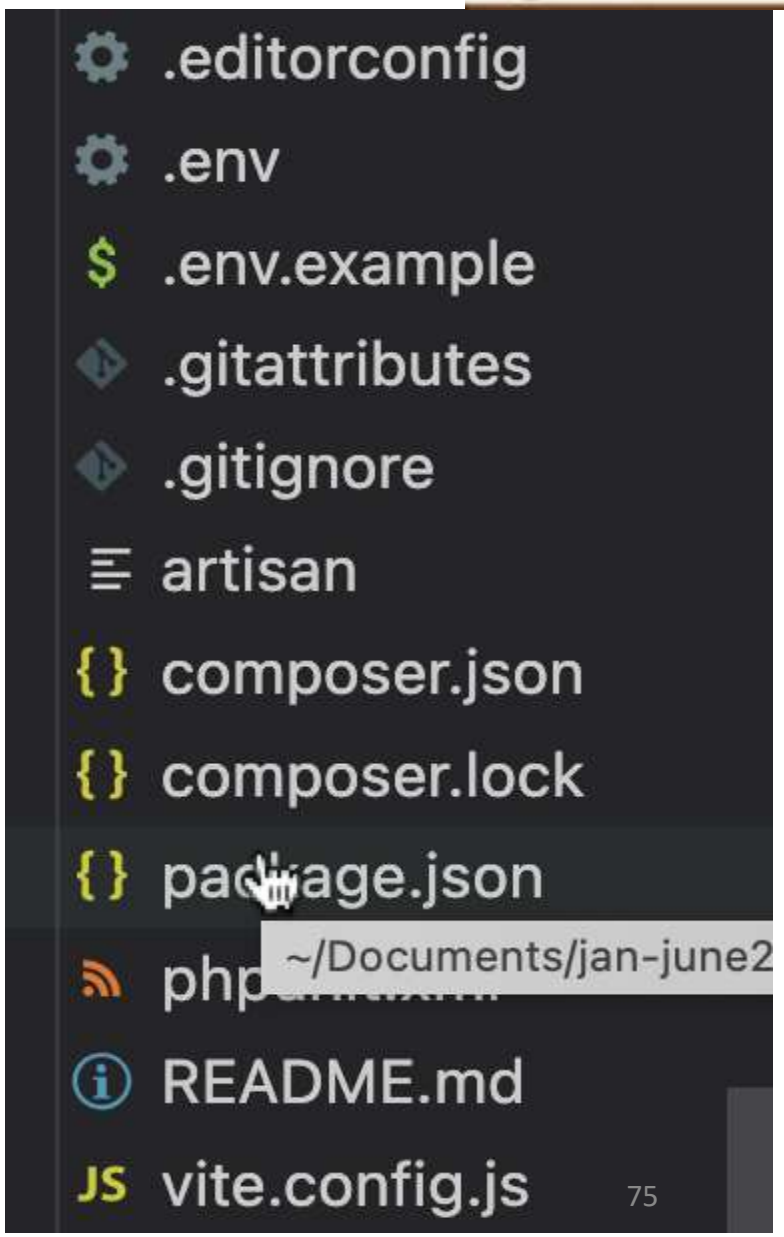
- Dependencies are organized in subdirectories based on their namespaces (e.g., vendor/symfony/, vendor/laravel/).
- It also contains the composer/ directory, which manages autoloading and metadata about the dependencies.

.editorconfig

This file is used to **maintain consistent coding styles** across different editors and IDEs in a project.

It allows you to **change settings without modifying code**

- Stores **database credentials**
- Keeps **API keys secure**
- Helps manage **different environments** (local, production)



.env

The `.env` **(environment file)** is used to store **configuration settings and sensitive data** for your application.

It allows you to **change settings without modifying code**

- Stores **database credentials**
- Keeps **API keys secure**
- Helps manage **different environments** (local, production)

.env.example

The `.env.example` file is a **template of the `.env` file**.

It contains **all required configuration keys**, but **without real/sensitive values**

- Helps other developers know **which environment variables are needed**
- Used when setting up a project for the first time
- Keeps **secrets safe** (no passwords or API keys inside)
- It looks almost like `.env`, but values are **dummy/empty**

.gitattributes

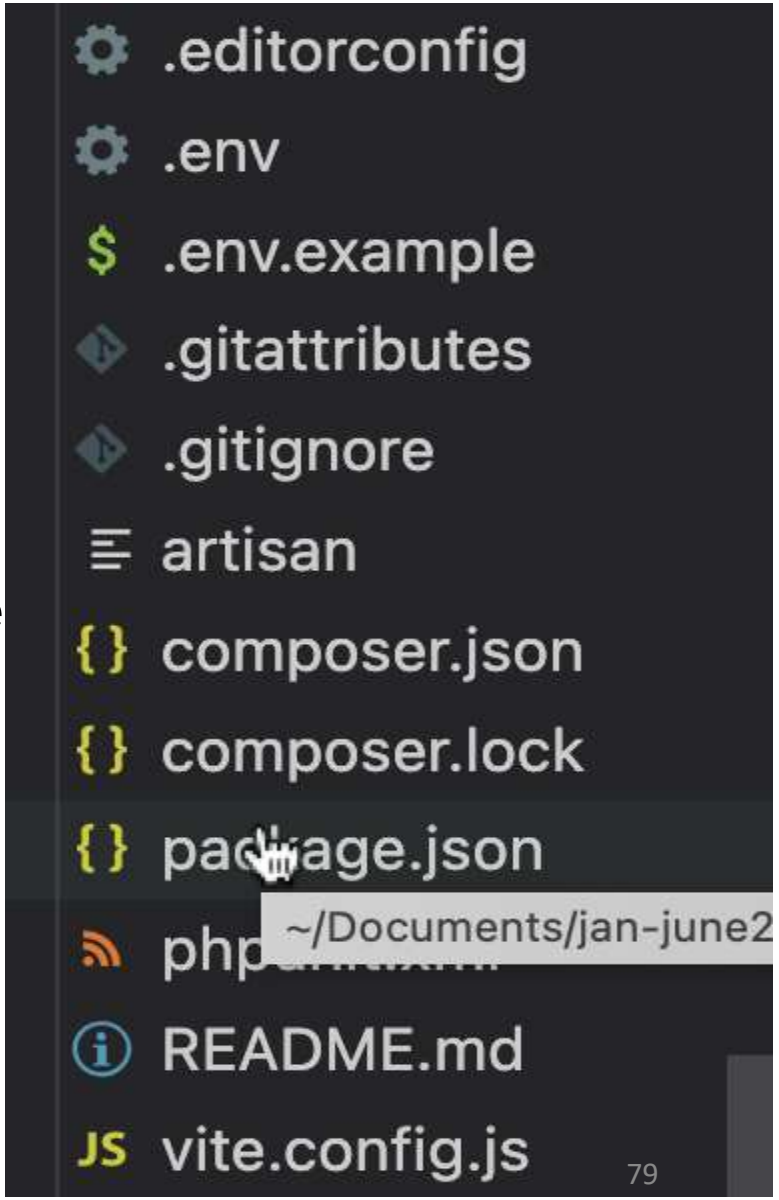
The `.gitattributes` file is used by **Git** to **control how files are handled in a repository**.

- Line endings
- File merging behavior
- Exporting files
- Language detection (GitHub)
- Ensures **consistent behavior across systems (Windows/Linux/Mac)**
- Prevents **merge conflicts**
- Helps GitHub correctly **detect languages**
- Controls what files are included in archives

composer.json

The `composer.json` file is the **main configuration file for Composer (PHP dependency manager)**.

- Which **libraries/packages** your project needs
- How to **autoload classes**
- Project **metadata & scripts**



composer.lock

- The **`composer.lock`** file is generated by Composer and stores the **exact versions of all installed dependencies** in your project.
- While `composer.json` defines *what you want*, **`composer.lock` records what you actually got installed**
- Exact **package versions**
- Dependency tree (sub-dependencies)
- Download sources (URLs)
- Hash for verification

package.json

`package.json` is a file that manages JavaScript dependencies and scripts in a project.

phpunit.xml

The **`phpunit.xml`** file is a **configuration file for PHPUnit**, which is used for **testing in PHP/Laravel**. It defines:

- Test environment settings
- Database config for testing
- Test directories

README.md

The **`README.md`** file is a **documentation file** that explains your project. It is written in **Markdown (.md)** format and is the **first file people see** on GitHub.

vite.config.js

The **`vite.config.js`** file is a **configuration file for Vite**, a modern frontend build tool used in Laravel.

It controls how your **CSS, JavaScript, and assets are compiled and served**

In every Laravel project, you will have two files, composer.json and composer.lock. What is the difference between them?

In composer.json you specify **what packages should be installed, and with what versions**.

```
{
    "require": {
        laravel/breeze": "^1.19",
    },
}
```

Now, **which exact version is installed at any moment**? Here comes the composer.lock file.

```
{
    // ...
    "name": "laravel/breeze",
    "version": "v1.19.1",
    "source": {
        "type": "git",
        "url": "https://github.com/laravel/breeze.git",
        "reference": "4bbb1ea3476901c4f5fc706f8d80e4eac31c3afb"
    },
    "dist": {
        "type": "zip",
        "url": "https://api.github.com/repos/laravel/breeze/zipball/4bbb1ea3476901c4f5fc70
```

Which directory contains the views (templates) used for rendering HTML pages in Laravel?

a) resources

b) public

c) app

d) routes

Which directory is used for storing files generated by the application, such as logs, cache, and session data?

a) resources

b) public

c) storage

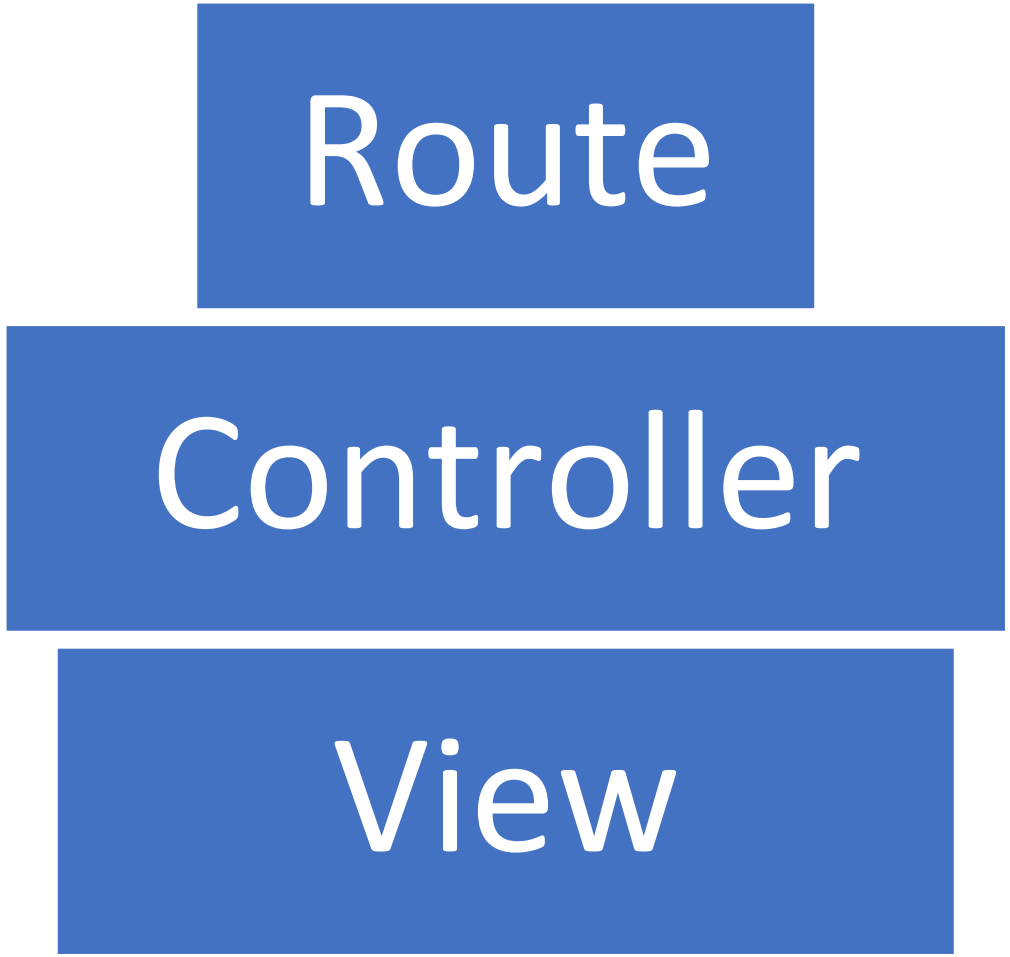
d) config

INT 221 - MVC PROGRAMMING

Unit 2

Laravel Request Lifecycle

Seems but not true
General Answer
But it is not True



Lifecycle

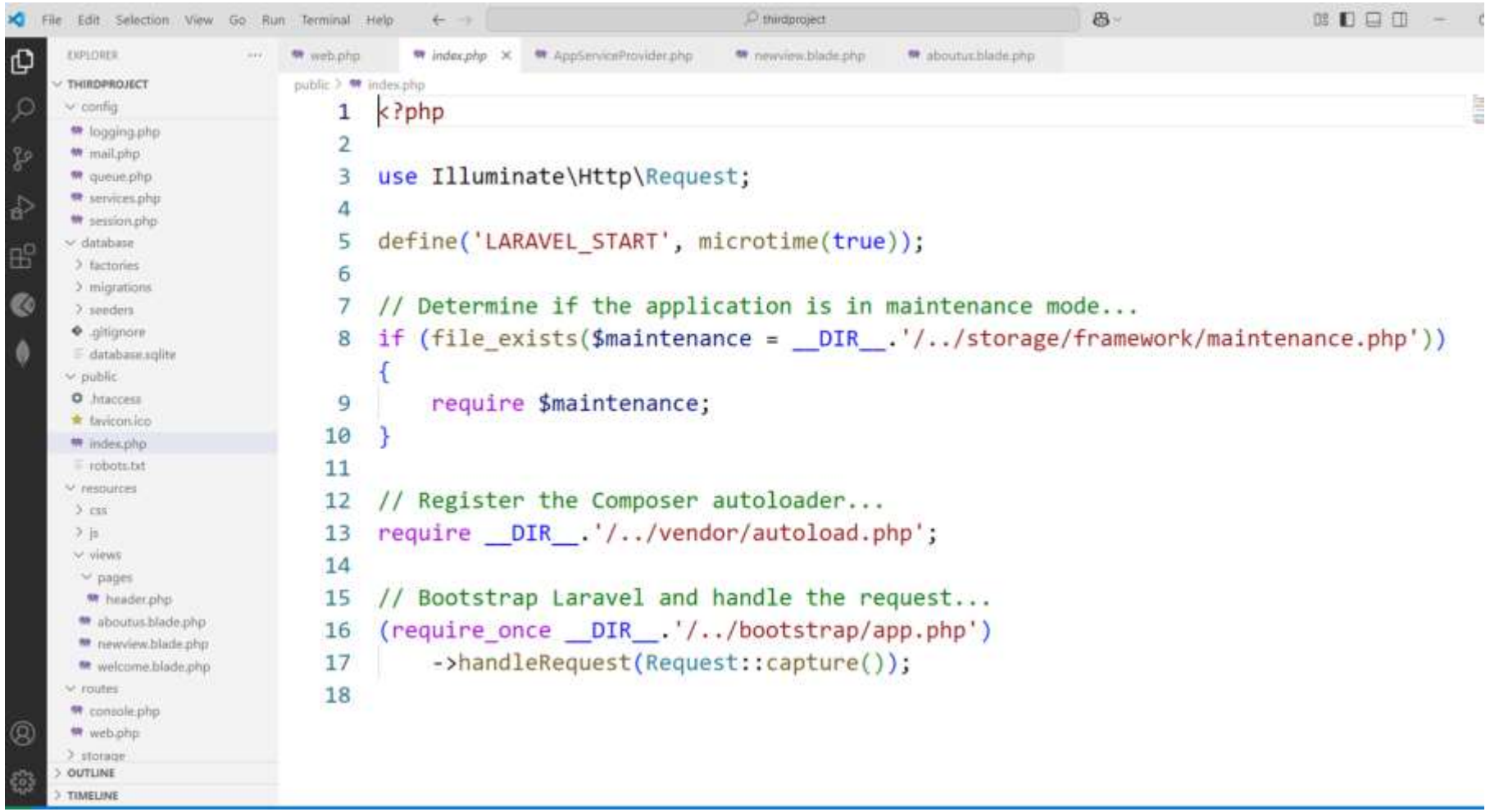
a **sequence of events and actions that occur during the execution of a request**. Each request goes through several stages, and Laravel provides various mechanisms to perform actions at different points in this lifecycle

Index.php – The ENTRY point

All **requests are directed to this file by your web server** (Apache / Nginx) configuration. The index.php file doesn't contain much code. Rather, it is a **starting point for loading the rest of the framework**.

The **index.php** file loads the Composer-generated **autoloader definition and** then **retrieves an instance of the Laravel application** from bootstrap/app.php.

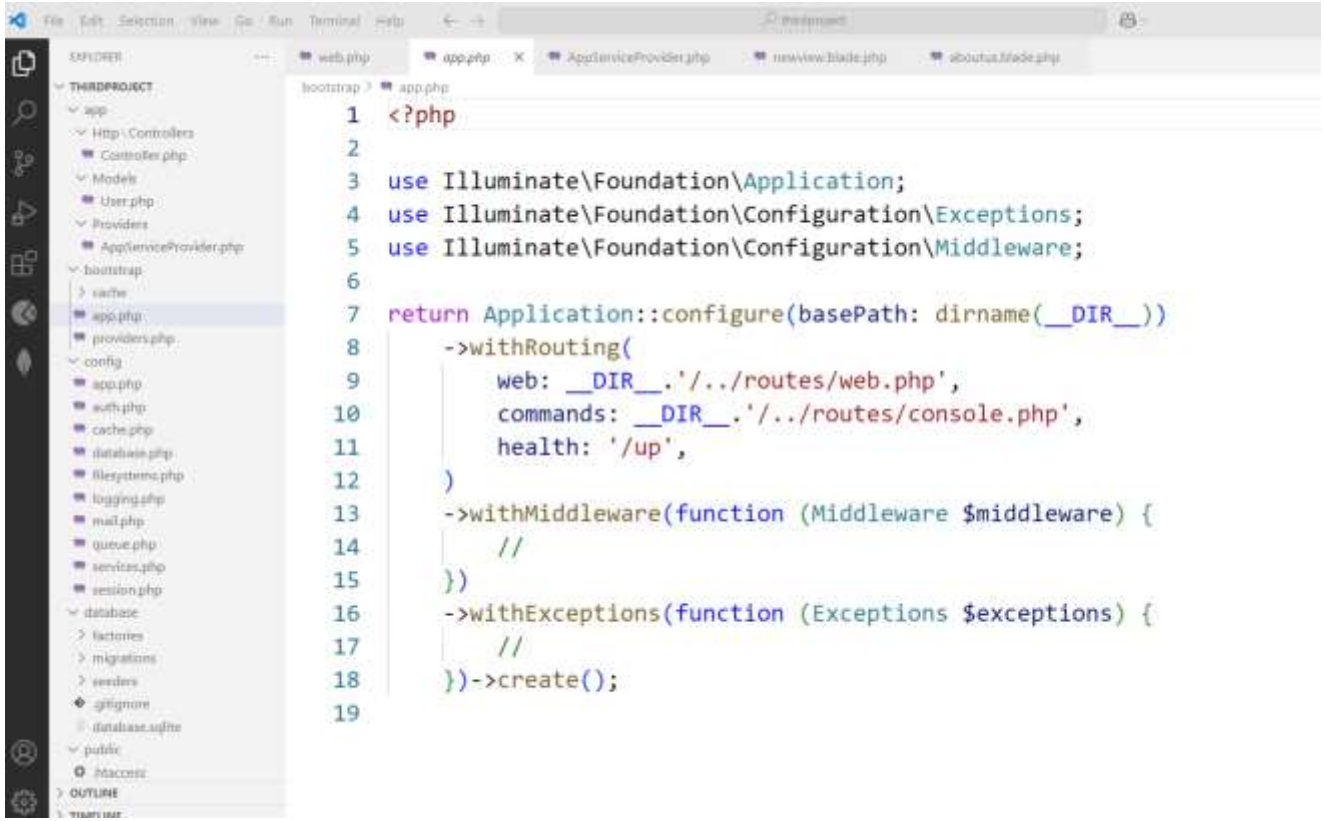
Structure of index.php file



Composer Autoloader

An autoloader in Laravel is like a **smart library organizer for your code**. It **automatically finds and loads the right files when you use different parts of your Laravel project**. This way, you don't have to search for and include files manually, making your coding life easier and more organized.

Bootstrap -> app.php
(Instances are made here)



```php
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        //
    })
    ->withExceptions(function (Exceptions $exceptions) {
        //
    })->create();

```

The bootstrap/app.php file sets up the Laravel application instance. It binds essential interfaces and prepares Laravel's service container.

Request is now sent to bootstrap/app.php

Two based on the request for routing:

- Web routing-> routes/web.php (hitting the URL via web browser)
- command -> routes/console.php

Service Provider

- the request is sent to Service Providers
- Implement core functionality e.g. Mailing, Sessions etc.

bootstrap/providers.php

- You can find a list of the user-defined or third-party service providers that your application is using in the bootstrap/providers.php file.

Service providers

- Service providers are responsible for bootstrapping all of the framework's various components, such as the database, queue, validation, and routing components, event management, session management.

Laravel comes with some default service providers:-

- App\Providers\AppServiceProvider.php
(in Laravel-11)
- App\Providers\AuthServiceProvider.php
- App\Providers\BroadcastServiceProvider.php
- App\Providers\EventServiceProvider.php
- App\Providers\RouteServiceProvider.php

Service Providers Loading (bootstrap/providers.php)

- Laravel loads all registered service providers to initialize core services like database, authentication, routing, etc.

Providers

- Config-> Appserviceprovider—to declare the global variables (custom stuff)

Routes (routes/web.php)

- Once the application has been bootstrapped and all service providers have been registered, the Request will be handed off to the **route**r for dispatching.
- The router will dispatch the request to a **route or controller, as well as run any route specific middleware.**
- **Laravel finds the appropriate route that matches the request URL.**

Middleware (using construct method)

- **Middleware** provides a convenient mechanism for filtering or examining HTTP requests entering your application.
- For example, Laravel includes a middleware that verifies if the user of your application is authenticated.
- If the user is not authenticated, the middleware will redirect the user to the login screen. However, if the user is authenticated, the middleware will allow the request to proceed further into the application.

Middleware Execution

- Authenticate users.
- Log requests.
- Validate input data.

Example Middleware (app/Http/Middleware/CheckAge.php)

```
public function handle($request, Closure $next)
{    if ($request->age < 18) {
        return response('Unauthorized', 403);
    }
return $next($request);
}
```

Controller

- In Laravel, controllers are found in the app/Http/Controllers directory.
- They help organize request handling logic by grouping related functions into classes.

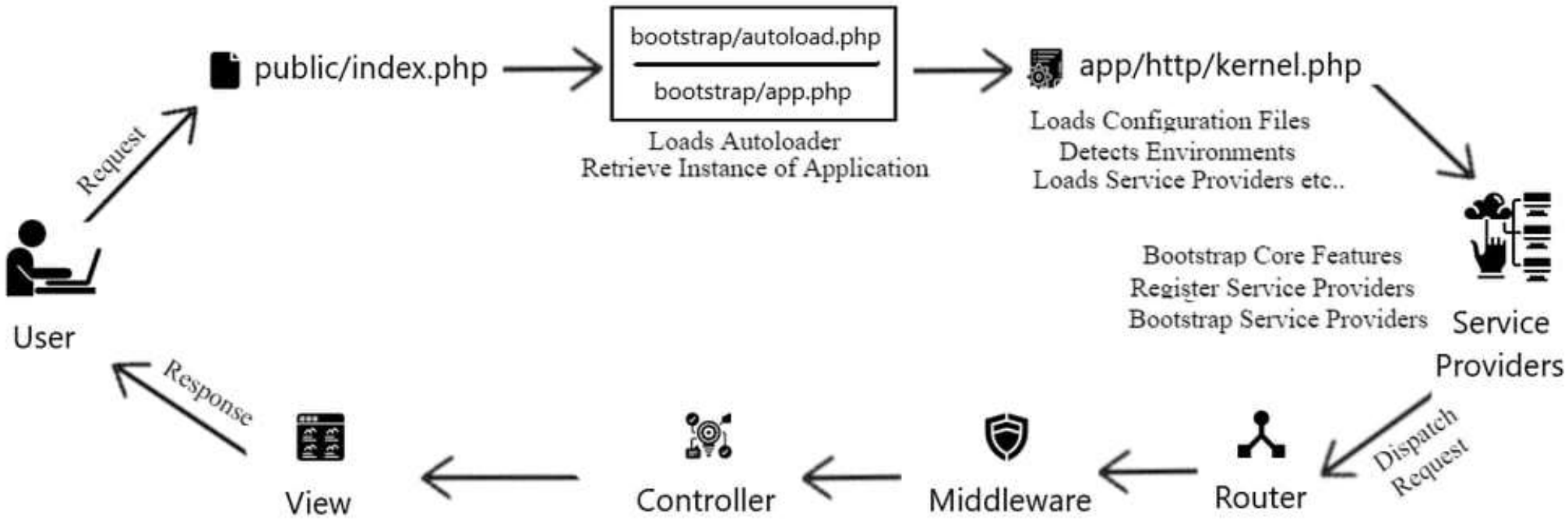
View

- If the route returns an HTML view, Laravel loads a Blade template.

(resources/views/home.blade.php)

```
<!DOCTYPE html>
<html>
<head>
 <title>
{{ $title }}
</title>
</head>
<body>
<h1>Hello, Laravel!</h1>
</body>
</html>
```

Laravel Request Life Cycle



By default laravel project runs on which PORT?

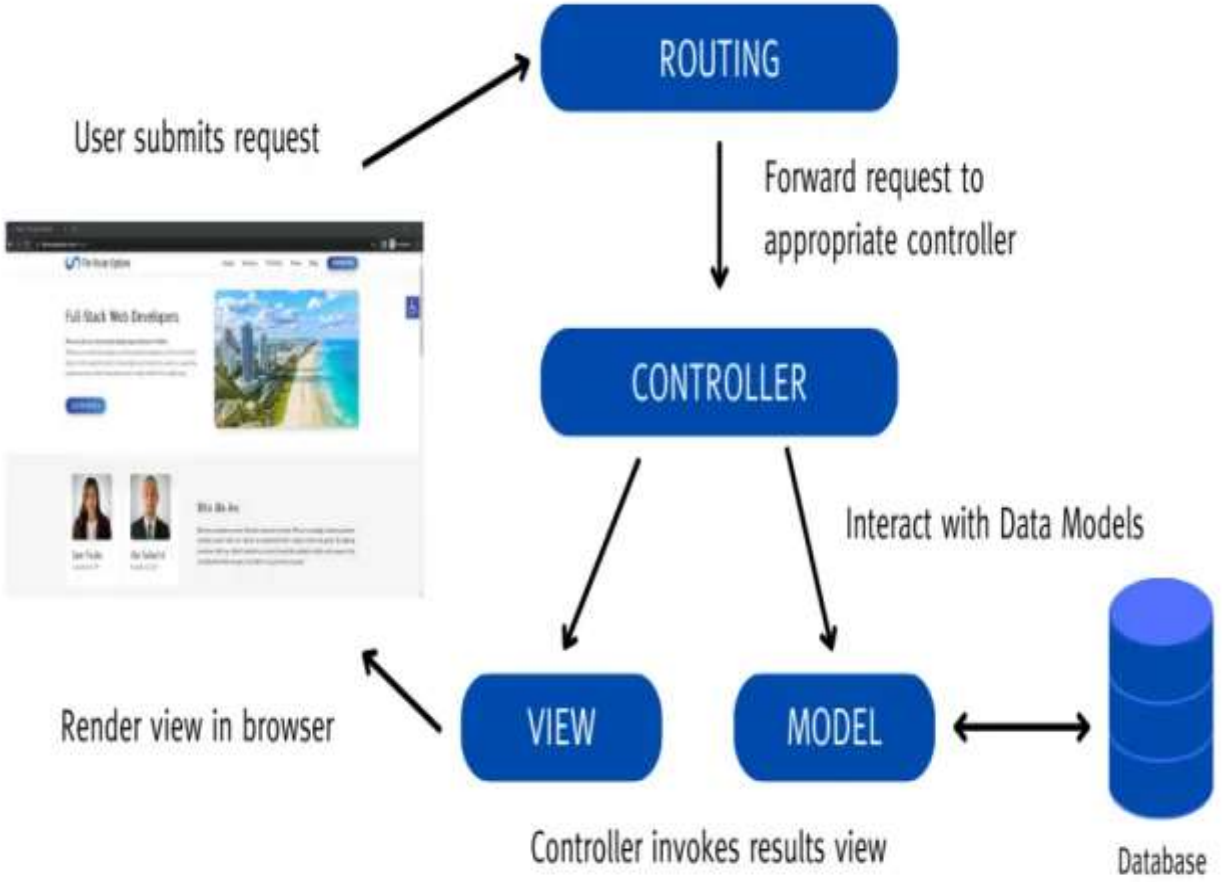
- 3000
- 5000
- 8000
- 7000

What's the command to check the current version of the Laravel framework?

- php artisan main --version
- php artisan --version status
- php artisan check --version
- php artisan --version

What is Routing?

The process of **directing incoming requests (usually based on URLs) to the appropriate code or handler** that will handle the request and generate a response.



Contd...

Routing is like a traffic director for web applications. When a user enters a URL in their browser or clicks a link, routing decides where that request should go within the application to generate the appropriate content or perform a specific action.

URL

URL stands for "Uniform Resource Locator." It's a reference or address used to access resources on the internet. In simpler terms, a URL is the **unique web address that you enter into a web browser to access a specific web page, file, or resource**.

By default one route is created by Laravel

```
use Illuminate\Support\Facades\Route;
Route::get('/', function () {
    return view('welcome');
});
```

In the above case, Route is the class which defines the static method get(). The get() method contains the parameters '/' and the function() closure. The '/' defines the root directory, and function() defines the functionality of the get() method.

Paamayim Nekudotayim ::

- It is a token that allows access to static, constant, and overridden properties or methods of a class. When referencing these items from outside the class definition, use the name of the class.
- It's possible to reference the class using a variable. The variable's value can not be a keyword (e.g. self, parent and static).

Do you know?

Paamayim Nekudotayim would, at first, seem like a strange choice for naming a double-colon. However, while writing the Zend Engine 0.5 (which powers PHP 3), that's what the Zend team decided to call it. It actually does mean double-colon - in Hebrew!

Route::get('/')

- Route::get('/') defines a route for a GET request to the root URL (/) of your application.
- The root URL typically refers to the base of your web application, e.g., http://yourdomain.com/.

Closure Function:

- The second parameter of Route::get is a closure (anonymous function). This function will execute when the route is accessed.

Returning a View:

- Inside the closure, return view('welcome'); tells Laravel to render and return a Blade view named 'welcome'. Laravel assumes the view file is located at resources/views/welcome.blade.php.

What is the purpose of the web.php routes file in Laravel?

- **a) It defines routes for web application routes and middleware.**
- b) It defines routes specifically for API endpoints.
- c) It defines routes for console commands and tasks.
- d) It defines routes for unit testing

Command to get the route list in Laravel

- **php artisan route:list**

In Laravel, where are the routes primarily defined?

- a) In the app/routes.php file
- b) In the routes folder within the app directory
- c) In the .env configuration file
- **d) In the web.php or api.php files within the routes directory**

HTTP Methods

GET, **POST**, **PUT**, **PATCH**, and **DELETE** are the five most common HTTP methods for retrieving from and sending data to a server.

The GET method

The GET method is used to retrieve data from the server. **This is a read-only method, so it has no risk of mutating or corrupting the data**. For example, if we call the get method on our API, we'll get back a list of all to-dos.

The POST method

The POST method sends data to the server and creates a new resource. The resource it creates is subordinate to some other parent resource. When a new resource is POSTed to the parent, the API service will automatically associate the new resource by assigning it an ID (new resource URI). In short, **this method is used to create a new data entry**

The PUT method

The PUT method is most often **used to update an existing resource**. If you want to update a specific resource you can call the PUT method to that resource URI with the request body containing the complete new version of the resource you are trying to update.

The PATCH method

The PATCH method is very similar to the PUT method because it also modifies an existing resource. The difference is that for the PUT method, **the request body contains the complete new version, whereas for the PATCH method, the request body only needs to contain the specific changes to the resource**, specifically a set of instructions describing how that resource should be changed, and the API service will create a new version according to that instruction.

The DELETE method

The DELETE method is used to delete a resource specified by its URI.

View in Laravel

- Views provide a convenient way to place all of our HTML in separate files.
- Views separate your controller / application logic from your presentation logic and are stored in the resources/views directory. A simple view might look something like this:

```
<!-- View stored in resources/views/greeting.blade.php -->
<html>
    <body>
        <h1>Hello, {{ $name }}</h1>
    </body>
</html>
```

- Since this view is stored at resources/views/greeting.blade.php, we may return it using the global view helper like so:

```
Route::get('/', function () {
    return view('greeting', ['name' => 'James']);
});
```

Creating & Rendering Views

- You may create a view by placing a file with the .blade.php extension in your application's resources/views directory.

- The .blade.php extension informs the framework that the file contains a Blade template.

- Blade templates contain HTML as well as Blade directives that allow you to easily echo values, create "if" statements, iterate over data, and more.

- command to create the view is:

  php artisan make:view user (here user is the name of the view).

- A view with name user.blade.php is created in resources/views.

Nested View Directories

- Views may also be nested within subdirectories of the resources/views directory.
- "Dot" notation may be used to reference nested views.
- For example, if your view is stored at resources/views/admin/profile.blade.php, you may return it from one of your application's routes / controllers like so:

```
return view('admin.profile', $data);
```

Passing Data To Views

- You may pass an array of data to views to make that data available to the view:

```
return view('greetings', ['name' => 'Victoria']);
```

- When passing information in this manner, the data should be an array with key / value pairs.

- After providing data to a view, you can then access each value within your view using the data's keys, such as <?php echo $name; ?>.

Contd…

- Inside Web.php

```
//Passing Data to View
Route::get('/test',function(){
    $name="akash";
    return view('test',['name'=>$name]);
});
```

- create the view test:
  in terminal: php artisan make:view test

- Inside test.blade.php

```
<h1>{{$name}}</h1>
```

Passing Data To Views (contd.)

- As an alternative to passing a complete array of data to the view helper function, you may use the with method to add individual pieces of data to the view.

- The with method returns an instance of the view object so that you can continue chaining methods before returning the view:

```
return view('greeting')
        ->with('name', 'Victoria')
        ->with('occupation', 'Astronaut');
```

```
Route::get('/test',function(){
    $name="akash";
    $city="Jalandhar";
    return view('test')
    ->with('name',$name)
    ->with('city',$city);
});
```

One More Shortcut

- In web.php

```
Route::get('/test',function(){
    $name="akash";
    $city="Jalandhar";
    return view('test')
    ->withName($name)
    ->withCity($city);
});
```

- In test.blade.php

```
<h1>Name : {{$name}} <br> City: {{$city}}</h1>
```

Suppose empty string is passed in city

- Instead of printing empty string you can also use conditional expression in blade template file and print some default value.

```
<h1>Name : {{$name}} <br>
    City: {{!empty($city)?$city:"Enter your City"}}
</h1>
```

Data passing methods

1. Using `with()` Method

return view('welcome')->with('name', 'Parveen');

In Blade (view file):

<h1>{{ $name }}</h1>

2. Using `compact()`

[$name = "Parveen", $age = 22];

return view('welcome', compact('name', 'age'));

In Blade:

<h1>{{ $name }}</h1>
{{ $age }}

`compact()` converts variables into an associative array:

['name' => $name, 'age' => $age]

Data passing methods

LOVELY PROFESSIONAL UNIVERSITY

3. Passing Data as Array

```
return view('welcome', [ 'name' => 'Parveen', 'age' => 22]);
```

4. Passing Multiple Data Using `with()`

```
return view('welcome')  ->with('name', 'Parveen') ->with('age', 22);
```

5. Passing Data from Route

```
Route::get('/test', function () {
    $students = [
        ['name' => 'Aman', 'roll' => 1],
        ['name' => 'Riya', 'roll' => 2]
    ];
    return view('students', compact('students'));
});
```

Data passing methods

7. Displaying Data in Blade (Loop)

```
@foreach($students as $student)
   <p>{{ $student['name'] }} - {{ $student['roll'] }}</p>
@endforeach
```

8. Passing Data to All Views (Global Data)

```
use Illuminate\Support\Facades\View;

View::share('appName', 'My Laravel App');
```

Now accessible in all views:

```
<h1>{{ $appName }}</h1>
```

9. Using `view()->with()` Helper

```
return view('welcome')->with([
   'name' => 'Parveen',
   'age' => 22
]);
```

What is the purpose of views in Laravel?

- A) To store database records.
- **B) To separate controller/application logic from presentation logic.**
- C) To define routes.
- D) To manage authentication.

Where are views typically stored in a Laravel application?

- A) In the public directory.
- B) In the app/views folder.
- **C) In the resources/views directory**.
- D) In the database folder.

Which file extension is used for Blade templates in Laravel views?

- A) .php
- B) .html
- **C) .blade.php**
- D) .view

In Blade templates, how can you access data that has been passed to a view?

- A) **By using the {{ data }} syntax.**
- B) By using echo data; syntax.
- C) By using $data directly.
- D) By using the @data directive.

What method can you use to add individual pieces of data to a view in Laravel, allowing you to chain multiple data additions?

- A) addData()
- B) includeData()
- **C) with()**
- D) attachData()

How can you share data with all views rendered by your Laravel application?

- A) By storing it in a database.
- B) By using global variables.
- **C) By using the share method within a service provider's boot method**.
- D) By passing it to each view individually.

Sharing Data With All Views

- Occasionally, you may need to share data with all views that are rendered by your application.
- Typically, you should place calls to the share method within a service provider's boot method.
- You are free to add them to the App\Providers\AppServiceProvider class or generate a separate service provider to house them:

Sharing Data With All Views(contd.)

```
?php

namespace App\Providers;

class AppServiceProvider extends ServiceProvider
{
public function boot()
    {
        view()::share('name', 'Pooja');
    }
}
on any view
<h2>User name is :{{$name}}<\h2>      // can be directly accessed
```

View Routes

- If your route only needs to return a view, you may use the Route::view method.
- This method provides a simple shortcut so that you do not have to define a full route or controller.
- The view method accepts a URI as its first argument and a view name as its second argument.

```
Route::view('/welcome', 'welcome');
```

Route Parameters

Required Parameters - Sometimes you will need to capture segments of the URI within your route. For example, you may need to capture a user's ID from the URL. You may do so by defining route parameters:

```
Route::get('/user/{id}', function ($id) {
    return 'User '.$id;
});
```

- You may define as many route parameters as required by your route.
- Route parameters are always encased within {} braces and should consist of alphabetic characters.
- Underscores (_) are also acceptable within route parameter names.

Contd...

Optional Parameters - Occasionally you may need to specify a route parameter that may not always be present in the URI. You may do so by placing a ? mark after the parameter name. Make sure to give the route's corresponding variable a default value:

```
Route::get('/user/{name?}', function ($name = null) {
    return $name;
});

Route::get('/user/{name?}', function ($name = 'John') {
    return $name;
});
```

Route Parameters Constraints

You may constrain the format of your route parameters using the family of ‘where’ methods on a route instance.

whereNumber Constraint:

The whereNumber constraint is used to ensure that a route parameter is a numeric value.

It is useful when you want to restrict a parameter to only accept numeric input.

```
Route::get('/user/{id}', function ($num) {
    return 'User '.$num;
})->whereNumber('id');
```

=> And -> are different

=> (Arrow or Fat Arrow)

- => is primarily used in association with key-value pairs, often seen in data structures like dictionaries, associative arrays, or objects in various programming languages.
- It is commonly used to assign values to keys or properties.

-> (Arrow or Member Access Operator)

- -> is used to access members (attributes or methods) of objects or structures in some programming languages, particularly in C-like languages such as C++ and PHP for object-oriented programming.
- It's used to access properties and methods of objects or structs.

whereAlpha

The whereAlpha constraint is used to ensure that a route parameter contains only alphabetic characters.

It restricts the parameter to accept only letters (A-Z and a-z).

```
Route::get('/user/{id}', function ($num) {
    return 'User '.$num;
})->whereAlpha('id');
```

whereAlphaNumeric

- The whereAlphaNumeric constraint is used to ensure that a route parameter contains only alphanumeric characters.
- It restricts the parameter to accept letters (A-Z and a-z) and numbers (0-9).

```
Route::get('/user/{id}', function ($num) {
    return 'User '.$num;
})->whereAlphaNumeric('id');
```

whereIn

- The whereIn constraint is used when you want to constrain a parameter's value to a specific set of values.
- It ensures that the parameter matches one of the specified values.

```
Route::get('/user/{id}', function ($num) {
    return 'User '.$num;
})->whereIn('id',['Attendance','Assignment']);
```

where

- The where method is a more general constraint that allows you to specify custom constraints using regular expressions or closures.
- It provides flexibility to define complex validation rules for a parameter.

```
Route::get('/user/{id}', function ($num) {
    return 'User '.$num;
})->where('id',"[0-9]+");
```

Handling Multiple Parameter Constraints

Suppose you have multiple parameters in URL, and you want to have different constrains for both, then one method for doing it is shown below.

```
Route::get('/user/{id}/{about}', function ($num) {
    return 'User '.$num;
})->where('id',"[0-9]+")->whereAlpha('about');
```

Route Parameters Global Constraint

- **Global Constraints** - In Laravel, you can define global route constraints by using the pattern method

- This allows you to apply constraints to all routes in your application. Global constraints can be useful for enforcing common validation rules on various route parameters throughout your application.

Route Parameters Global Constraint

- Here's how you can define a global route constraint for all routes:
- Open App/Providers/AppServiceProvider.php.

```
use Illuminate\Support\Facades\Route;
public function boot()
{
    parent::boot();
   Route::pattern('id', '[0-9]+');
}
```

- In the above code, we're using the Route::pattern method to specify a regular expression pattern [0-9]+ for the id parameter. This pattern requires that any id parameter in your routes must consist of numeric characters.

What is Laravel routing primarily used for?

- A. Database management
- **B. Handling HTTP requests**
- C. Front-end development
- D. Server configuration

In Laravel, where are the routes typically defined?

- A. In the .env file
- B. In the config/routes.php file
- **C. In the routes/web.php and routes/api.php files**
- D. In the resources/views directory

Which of the following is used to pass parameters in a Laravel route?

- A. <param>
- B. [param]
- **C. {param}**
- D. (param)

Which artisan command is used to list all registered routes in a Laravel application?

- A. php artisan serve
- B. php artisan list
- C. php artisan routes
- **D. php artisan route:list**

What is the primary purpose of Laravel routing constraints?

- A. To define routes in the web.php file
- B. To restrict access to certain routes
- **C. To specify rules that route parameters must follow**
- D. To create dynamic routes

Which built-in constraint is used to ensure that a route parameter is numeric?

- **A. whereNumber**
- B. whereInteger
- C. whereNumeric
- D. whereDigit

Returning Response in Laravel

- The aim of the response is to provide the client with the resource it requested or inform the client that the action it requested has been carried out; or else, to inform the client that an error occurred in processing its request.

```
Route::get('/greet', function(){
return "Hello World";
});
```

If we write "abc," it will open at this URL.

```
Route::get('/abc', function(){
return "Hello World";
});
```

Creating Responses

- **Strings & Arrays** - All routes and controllers should return a response to be sent back to the user's browser.
- Laravel provides several different ways to return responses.
- The most basic response is returning a string from a route or controller. The framework will automatically convert the string into a full HTTP response:

```
Route::get('/', function () {
    return 'Hello World';
});
```

- In addition to returning strings from your routes and controllers, you may also return arrays.
- The framework will automatically convert the array into a JSON response:

```
Route::get('/', function () {
    return [1, 2, 3];
});
```

Creating Responses (contd.)

- **Response Objects –**

```
Route::get('/home', function () {
    return response('Hello World', 200);
});
```

Attaching Headers To Responses

- You may use the header method to add a series of headers to the response before sending it back to the user:

```
return response($content)
            ->header('Content-Type', 'text/html')
```

View Responses

- If you need control over the response's status and headers but also need to return a view as the response's content, you should use the view method:

```
return response()
        ->view('hello')
        ->header('Content-Type', 'text/html');
```

JSON Responses

- The json method will automatically set the Content-Type header to application/json, as well as convert the given array to JSON using the json_encode PHP function.

```
return response()->json([
    'name' => 'Abigail',
    'state' => 'CA',
]);
```

Do you know?

Douglas Crockford was the first to define and popularise the JSON format. In April 2001, Douglas Crockford and Chip Morningstar sent the first JSON message.

What is the primary goal of the response in Laravel?

- A) To confuse the client
- B) To provide irrelevant data
- **C) To provide the requested resource or inform about the outcome**
- D) To prevent the client from accessing resources

How can you return a string as a response in a Laravel route or controller?

- **A) return 'Hello World';**
- **B) return response('Hello World');**
- C) echo 'Hello World';
- D) Response::send('Hello World');

Which method should be used if you want to return view along with control over status and headers in Laravel?

- A) return view('hello')
- **B) return response()->view('hello')**
- C) return respo('hello')->view()
- D) return view('hello')->re()

What does the json method do in Laravel's response?

- A) Converts the response to plain text
- **B) Sets the response content to JSON format**
- C) Converts the response to XML
- D) Attaches JavaScript code to the response

Subrouting

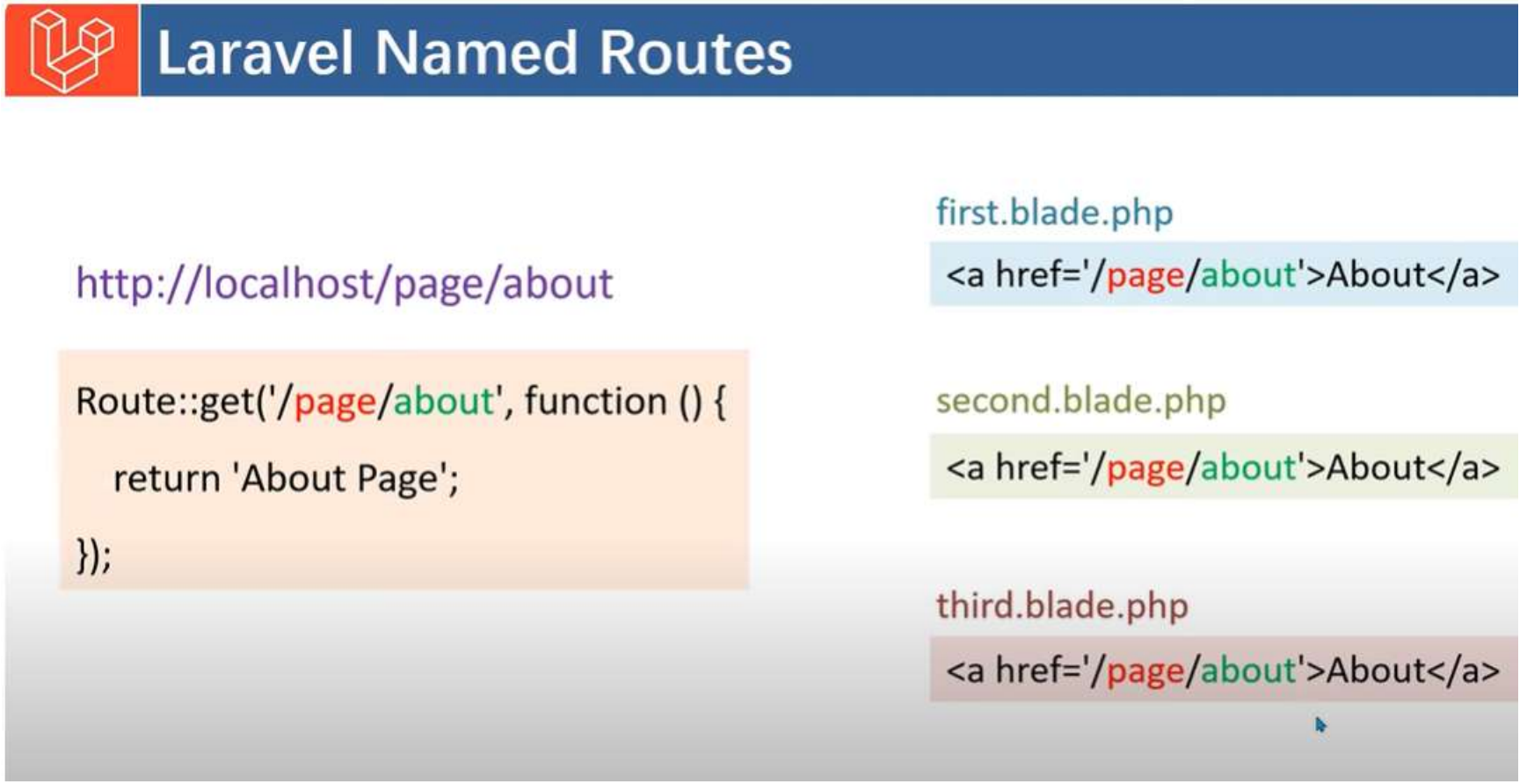
```
Route::get('/home/page',function(){
return "<h1 style='color:red'> This is sub routing
</h1>";
});
```

Named Routes

```
Route::get('/name/lpu/about',function(){
return view("aboutus");
        });
```

Upon going on http://127.0.0.1:8000/name/lpu/about, it will open aboutus.blade.php

If I want to use the same route in multiple blade files



Suppose I need to change the Route name

http://localhost/page/about

```
Route::get('/page/about-us', function () {
    return 'About Page';
});
```

first.blade.php

```
<a href='/page/about'>About</a>
```

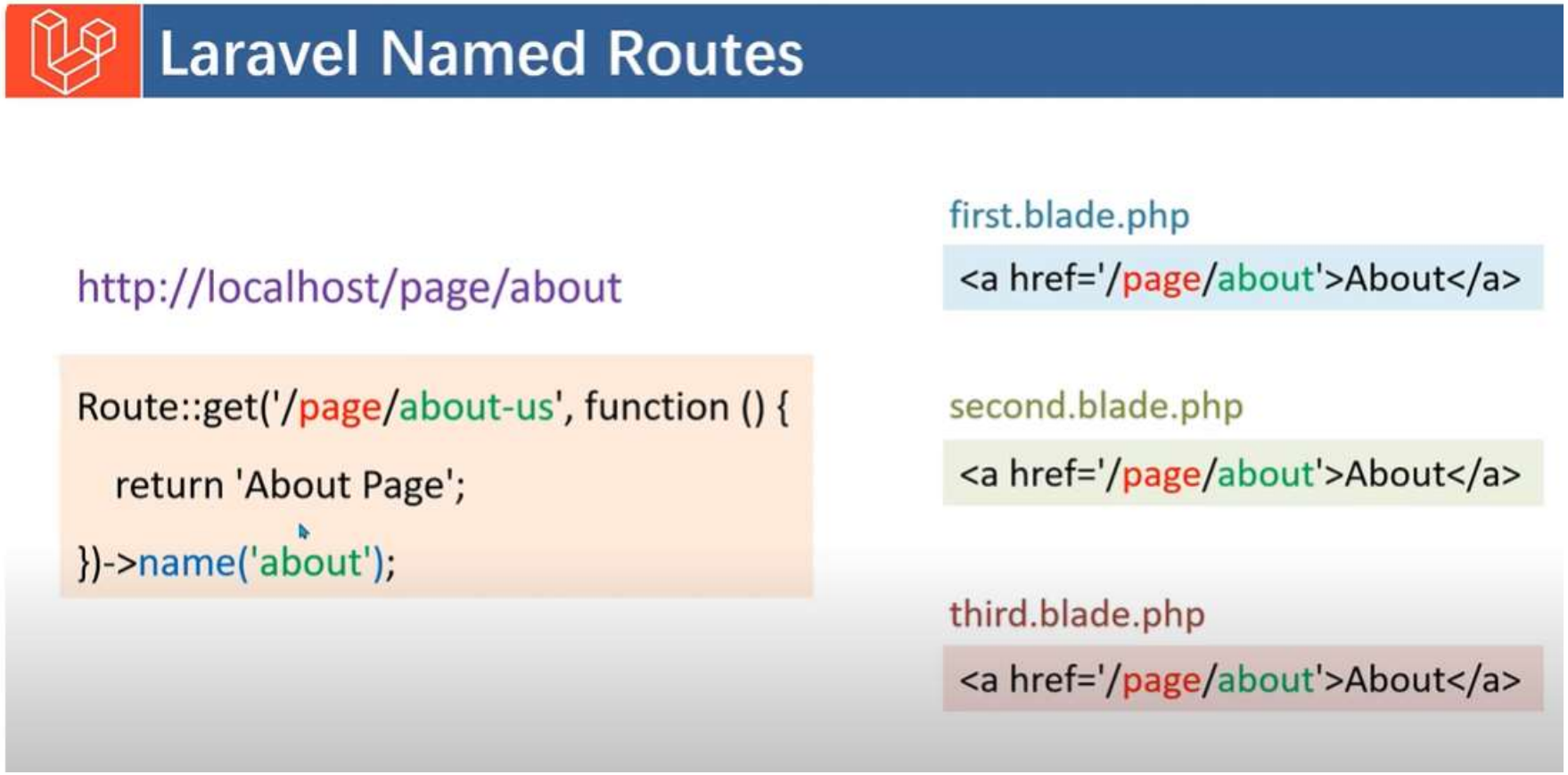
second.blade.php

```
<a href='/page/about'>About</a>
```

third.blade.php

```
<a href='/page/about'>About</a>
```

Add name('about'); in Routes



Add route function in blade template

http://localhost/page/about

```
Route::get('/page/about-us', function () {
    return 'About Page';
})->name('about');
```

first.blade.php

```
<a href='{{ route('about') }}'>About</a>
```

second.blade.php

```
<a href='{{ route('about') }}'>About</a>
```

third.blade.php

```
<a href='{{ route('about') }}'>About</a>
```

Giving the name to route with name(‘about’);
Use route (‘about’) in Blade template

Benefits of named route

- No need to change the <a> tag
- Instead of using parent and sub-route naming (which is very large), we use named routes.

Anchor Tag

```
<h1> About Us </h1>
<h1> {{$name}} </h1>
<a href="/"> Welcome Page </a>
```

The Route for / must be declared in web.php

```
Route::get('/', function () { // URL to open
    return view('welcome');
});
```

Output-On Clicking Welcome Page, http://127.0.0.1:8000 will get open

Redirection

Redirection in Laravel is used to send users from one URL/route to another. It’s very common after form submission, login, logout, or when access is restricted.

- The simplest method is to use the global redirect helper:

```
Route::get('/dashboard', function () {
    return redirect('home/dashboard');
});
```

- **Redirect to previous page**

```
return redirect()->back();
```

- **Redirect to Named Routes**

```
Route::get('/dashboard', function () {
   return "Dashboard";
})->name('dashboard');
```

Redirect

```
return redirect()->route('dashboard');
```

- **Redirect to Controller Action**

```
return redirect()->action([UserController::class, 'index']);
```

- **Redirect with Data (Flash Session)**

Pass success message

```
return redirect('/home')->with('success', 'Login successful!');
```

Access in Blade

```
@if(session('success'))
  <p>{{ session('success') }}</p>
@endif
```

- **Redirect with Input Data**

```
return redirect()->back()->withInput();
```

Keeps old form values

In Blade

```
<input type=”text" name="name" value="{{ old('name') }}">
```

- **Redirect with Errors**

  return redirect()->back()->withErrors(['name' => 'Name is required']);

  Used in validation

- **Redirect with HTTP Status Code**

  return redirect('/home', 301);

  - `302` → Temporary (default)
  - `301` → Permanent

- **Redirect to External URL**

  return redirect()->away('https://google.com');

  Bypasses Laravel URL validation

- **Redirect in Routes**

```
Route::get('/old-page', function () {
   return redirect('/new-page');
});
```

- **Redirect using Route Shortcut**

```
Route::redirect('/old', '/new', 301);
```

- **Redirect to fallback**

```
Route::fallback('/route', function () {
   return “page not existing;
});
```

// Here for the route, pass the route value that is not existing in the project, and then instead of 404/not found, it will display this return page not existing.

INT 221 - MVC PROGRAMMING

Unit 3

Controller

- In Laravel, a controller is a crucial component used to handle HTTP requests and define the application's response logic. Controllers serve as an intermediary between routes and the actual business logic of your application.

- Instead of defining all of your request handling logic as closures in your route files, you may wish to organize this behavior using "controller" classes.

- Controllers can group related request handling logic into a single class.

Controller

- For example, a UserController class might handle all incoming requests related to users, including showing, creating, updating, and deleting users.

- By default, controllers are stored in the app/Http/Controllers directory.

- A **Controller** handles request logic and connects:
- Route → Controller → View/Response

- Keeps code clean (follows MVC pattern)
- Avoids writing logic in routes

php artisan make:controller StudentController

File created at: app/Http/Controllers/StudentController.php

Basic Controller Structure

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class StudentController extends Controller
{
   public function index()
   {
       return "All Students";
   }
}
```

index() → method (action)

Controller with Route

- In routes/web.php

  use App\Http\Controllers\StudentController;
  Route::get('/students', [StudentController::class, 'index']);

  make sure project is running: php artisan serve

- Now open:

  http://localhost:8000/students

- Output:

  All Students

Passing Data from Controller to View

- **Controller:**

```
public function index()
{
   $students = [
       ['id' => 1, 'name' => 'Ravi'],
       ['id' => 2, 'name' => 'Aman']
   ];
   return view('students', compact('students'));
}
```

- **View (`resources/views/students.blade.php`)**

```
@foreach($students as $student)
   <p>{{ $student['name'] }}</p>
@endforeach
```

- Data passed using `compact()`

Dynamic Route + Controller (Single Student)

- **Route:** Route::get('/student/{id}', [StudentController::class, 'show']);
- **Controller:**

```
public function show($id)
{
   $students = [      ['id' => 1, 'name' => 'Ravi'],
                                          ['id' => 2, 'name' => 'Aman']    ];
   foreach ($students as $student) {
       if ($student['id'] == $id) {
           return view('student-detail', compact('student'));
       }
   }
   return "Student not found";
}
```

Fetch specific student using ID: like 127.0.0.1:8000/student/1

Multiple Methods in Controller

```
class StudentController extends Controller
{
        public function index()
        {               return "All Students";                }

        public function create()
        {               return "Create Student Form";          }

        public function store(Request $request)
        {                       return "Student Saved";                 }
}
```

Common CRUD methods

Resource Controller

Creation: php artisan make:controller StudentController --resource

- Auto-generates:

```
index()     // show all
create()    // form
store()     // save
show()      // single
edit()      // edit form
update()    // update
destroy()   // delete
```

- **Route:**

  Route::resource('students', StudentController::class);

  Automatically creates all routes

Middleware

Middleware is a mechanism that acts as a **filter for HTTP requests** entering your application. It allows you to inspect, modify, allow, or deny requests **before they reach the controller**, and also modify responses **before they are sent back to the user**. Middleware is a layer between **request and response** that checks conditions like authentication, validation, or permissions before allowing access to routes.

Working Flow

Request → Middleware → Controller → Response → Middleware → Browser

`age.check` → **apply on complete project (global middleware)**
`country.check` → **apply on specific route + route group**

STEP 1: Create Middleware

Run commands:

php artisan make:middleware Agecheck
php artisan make:middleware Countrycheck

Middleware

STEP 2: Write Agecheck (GLOBAL)

`app/Http/Middleware/Agecheck.php`

```
<?php
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
class Agecheck{
   public function handle(Request $request, Closure $next){
       $age = $request->age;
       if ($age < 18) {
           return "Not allowed (Age < 18)";
       }
       return $next($request);
   }
}
```

Middleware

LOVELY PROFESSIONAL UNIVERSITY

STEP 3: Write Countrycheck (ROUTE)

`app/Http/Middleware/Countrycheck.php`

```
<?php
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
class Countrycheck{
   public function handle(Request $request, Closure $next){
       $country = $request->country;
       if ($country != "India") {
           return "Access denied (Only India allowed)";
       }
       return $next($request);
   }
}
```

Middleware

STEP 4: Register Middleware

`bootstrap/app.php`

```
use Illuminate\Foundation\Configuration\Middleware;

->withMiddleware(function (Middleware $middleware) {
    //Global middleware (runs on all routes)
    $middleware->append(\App\Http\Middleware\Agecheck::class);

    // Alias for route-specific middleware
    $middleware->alias([
        'country.check' => \App\Http\Middleware\Countrycheck::class,
    ]);
});
```

Middleware

STEP 5: Define Routes

`Routes/web.php`

1. Normal Route (Only Agecheck applies automatically)

```
Route::get('/home', function () {
   return "Home Page";
});
```

URL: http://127.0.0.1:8000/home?age=20

2. Single Route with Countrycheck

```
Route::get('/profile', function () {
   return "User Profile";
})->middleware('country.check');
```

URL: http://127.0.0.1:8000/profile?age=20&country=India

/home?age=15

Output:

Not allowed (Age < 18)

/profile?age=20&country=USA

Output:

Access denied (Only India allowed)

/profile?age=20&country=India

Output:

User Profile

Middleware

Route Group with Countrycheck

```
Route::middleware(['country.check']) ->group(function () {
        Route::get('/dashboard', function () {
            return "Dashboard";
        });

        Route::get('/settings', function () {
            return "Settings";
        });
});
```

URLs:

http://127.0.0.1:8000/dashboard?age=20&country=India

http://127.0.0.1:8000/settings?age=20&country=India

Controller Middleware

- Controller middleware is used to **filter requests before they reach controller methods**.
  - Applied inside controller
  - Controls access (auth, role, etc.)
  - Runs **before method execution**

- Middleware registered in `bootstrap/app.php`
- Used in controller via `$this->middleware()`
- **Create Middleware:** php artisan make:middleware CheckRole
- middleware created at: app/Http/Middleware/CheckRole.php

Write Middleware Logic

```
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
class CheckRole
{                   public function handle(Request $request, Closure $next)
                    {           // Example logic
                    if ($request->user() && $request->user()->role === 'admin') {
                                        return $next($request);
                                }
                return redirect('/home')->with('error', 'Unauthorized Access');
                    }
}
```

- Allows only admin
- Others redirected

Register Middleware Logic

Open: `bootstrap/app.php`

```
->withMiddleware(function ($middleware) {
    $middleware->alias([
        'role' => \App\Http\Middleware\CheckRole::class,
    ]);
})
```

✓ Alias name = `role`

Create Controller

php artisan make:controller AdminController

Apply Middleware in Controller

```
namespace App\Http\Controllers;

class AdminController extends Controller
{
        public function __construct()
        {               $this->middleware('role');              }

        public function dashboard()
        {               return "Admin Dashboard";               }
}
```

- Middleware applied to all methods

Define Route

```
use App\Http\Controllers\AdminController;
Route::get('/admin', [AdminController::class, 'dashboard']);
```

Demonstration

Case 1: Admin user

$request->user()->role = 'admin';

- Output:

  Admin Dashboard

Case 2: Normal user

$request->user()->role = 'user';

- Redirect:

  /home

- Message:

  Unauthorized Access

Controller Middleware Variations

LOVELY PROFESSIONAL UNIVERSITY

1. Apply to Specific Methods

```
public function __construct(){
        $this->middleware('role')->only('dashboard');
}
```

2. Exclude Methods

```
public function __construct(){
        $this->middleware('role')->except(['index']);
}
```

3. Multiple Middleware

```
$this->middleware(['auth', 'role']);
```

4. Middleware with Parameter

```
$this->middleware('role:admin');
```

Contd…

5. Inline Middleware

```
public function __construct(){
   $this->middleware(function ($request, $next) {
       if (auth()->check()) {
           return $next($request);
       }

       return redirect('/login');
   });
}
```

FLOW

Request → Route → Controller → Middleware → Method → Response

BLADE TEMPLATES

- Blade is the simple, yet powerful templating engine that is included with Laravel.

- Blade template files use the .blade.php file extension and are typically stored in the resources/views directory.

- You may display data that is passed to your Blade views by wrapping the variable in curly braces. For example, given the following route:

```
Route::get('/', function () {
    return view('welcome', ['name' => 'Samantha']);
});
```

- You may display the contents of the name variable like so:

```
Hello, {{ $name }}.
```

Blade Directives

- Blade directives are short PHP snippets that are used to extend the functionality of Blade templates. They are prefixed with an '@' symbol and can be used to perform a variety of tasks, such as:

- **Conditional logic:** The @if and @else directives can be used to conditionally render output.

- **Loops:** The @foreach and @forelse directives can be used to loop through collections of data.

- **Including other views:** The @include directive can be used to include the contents of another view file.

- **Rendering expressions:** The @php and @endphp directives can be used to render PHP expressions.

Loops

- Blade provides simple directives for working with PHP's loop structures.

```
@foreach ($users as $user)
    <p>This is user {{ $user}}</p>
@endforeach

@forelse ($users as $user)
    <li>{{ $user}}</li>
@empty
    <p>No users</p>
@endforelse
```

If Statements

- You may construct if statements using the @if, @elseif, @else, and @endif directives.

```
@if (count($records) === 1)
    I have one record!
@elseif (count($records) > 1)
    I have multiple records!
@else
    I don't have any records!
@endif
```

If Statements (contd.)

- In addition to the conditional directives already discussed, the @isset and @empty directives may be used as convenient shortcuts for their respective PHP functions:

```
@isset($records)
    // $records is defined and is not null...
@endisset

@empty($records)
    // $records is "empty"...
@endempty
```

Switch Statements

- Switch statements can be constructed using the @switch, @case, @break, @default and @endswitch directives:

```
@switch($i)
    @case(1)
        First case...
        @break

    @case(2)
        Second case...
        @break

    @default
        Default case...
@endswitch
```

Now Some Practical of blade directives

- Step 1 – Make a Controller using Artisan
- **php artisan make:controller UserController**

- Step 2 – Make a route in web.php and don't forget to include the UserController in web.php
- `Route::get('/profile', [UserController::class, 'showProfile']);`

- Step 3, Now in UserController class let's define the function showProfile()

```
public function showProfile() {
        $username = 'Mr Stark';
        $isAdmin = true;
        $items = ['Item 1', 'Item 2', 'Item 3'];
        return view('profile', compact('username', 'isAdmin',
'items'));
 }
```

Step 4- define a view in resources/view folder ‘profile.blade.php’

```
<!DOCTYPE html>
    <html>
        <head>
                    <title>User Profile</title>
        </head>
        <body>
           <h1>Welcome to your profile, {{ $username }}</h1>
                    @if($isAdmin)
                    <p>You are an administrator.</p>
                    @else
                    <p>You are not an administrator.</p>
                    @endif
                    <ul>
              @foreach($items as $item)
             <li>{{ $item }}</li>
             @endforeach
          </ul>
```

```
<hr>
        <h2>Additional Blade Directives:</h2>
        {{-- @unless directive --}}
        @unless($isAdmin)
                    <p>You don't have admin privileges.</p>
                @endunless
     {{-- @empty directive --}}
        @empty($additionalData)
          <p>Additional data is empty.</p>
         @endempty
    {{-- @for directive --}}
    <h3>Looping with for</h3>
        <ul>
     @for($i = 1; $i <= 3; $i++)
        <li>Iteration {{ $i }}</li>
     @endfor
</ul>
```

```
            @for($i = 1; $i <= 3; $i++)
            <li>Iteration {{ $i }}</li>
        @endfor
    </ul>

    {{-- @include directive --}}
    {{--<h3>Including Sub-Views:</h3>
    @include('partials.footer')--}}

    {{-- Blade Comments --}}
    {{-- This is a Blade comment --}}

</body>
</html>
```

What file extension is typically used for Blade template files in Laravel?

- a) .php
- **b) .blade.php**
- c) .tpl
- d) .view

- **How can you display a variable named $name in a Blade template?**
  - a) Hello, $name.
  - b) Hello, {!! $name !!}.
  - **c) Hello, {{ $name }}.**
  - d) Hello, {{$naame}}.

Which symbol is used to prefix Blade direc Laravel?

a) #
b) $
c) !
d) @

How do you create a loop in a Blade template to iterate through an array called $users?

a) @for ($user in $users)
b) @foreach ($user as $users)
c) @loop ($users as $user)
d) @foreach ($users as $user)

Which Blade directive is used to check if a variable is set and not null?

a) @isset

b) @empty

c) @ifset

d) @nullcheck

Which Blade directive is used for executing a loop with a defined number of iterations?

a) @while

b) @loop

c) @for

d) @foreach

Template Inheritance

Template Inheritance in Laravel is a **Blade feature** that allows you to create a **master layout** and reuse it across multiple pages.

- It helps avoid code repetition (DRY principle)
- It means:
  - Create one **main layout (parent)**
  - Other pages (**child views**) extend it

Master Layout → Child Views → Final Output

STEP 1: Create Master Layout

`resources/views/layouts/app.blade.php`

```
<!DOCTYPE html>
<html>
        <head>
                <title>@yield('title')</title>
        </head>
        <body>
                <header>
                        <h1>My Website</h1>
                </header>
                <nav>
                        <a href="/">Home</a>
                        <a href="/about">About</a>
                </nav>
                <hr>
                <div>
                         @yield('content')
                </div>
                <footer>
                        <p>© 2026 My Website</p>
                </footer>
        </body>
</html>
```

`@yield()` → placeholder for child content

STEP 2: Create Child View

`resources/views/home.blade.php`

```
@extends('layouts.app')

@section('title')
  Home Page
@endsection

@section('content')
  <h2>Welcome to Home Page</h2>
@endsection
```

`@extends` → inherit layout

`@section` → fill content

STEP 3: Route + Controller

```
Route::get('/', function () {
  return view('home');
});
```

OUTPUT

Final rendered page:

```
My Website
----------
Welcome to Home Page
----------
© 2026 My Website
```

IMPORTANT BLADE DIRECTIVES

1. **`@extends`**

   @extends('layouts.app')

   Inherit layout

2. **`@section`**

   @section('content')
   Data here
   @endsection

   Define section content

3. **`@yield`**

   @yield('content')

   Display section

4. **`@include`**

   @include('partials.header')

   Include reusable parts (header/footer)

Secure Routing

In **Laravel**, *secure routes* means protecting routes so only authorized or authenticated users can access them. This is mainly done using **middleware**, especially authentication and authorization middleware.

Secure routes are routes that:

- Require **login (authentication)**
- Restrict access based on **roles/permissions**
- Prevent unauthorized users from accessing sensitive pages

Using Middleware for Secure Routes

Laravel provides built-in middleware like:

- `auth` → only logged-in users
- `verified` → email verified users
- `throttle` → rate limiting
- custom middleware → role-based security

Example 1: Secure Route with Authentication

```
use Illuminate\Support\Facades\Route;

Route::get('/dashboard', function () {
   return "Welcome to dashboard";
})->middleware('auth');
```

Only logged-in users can access `/dashboard`

Example 2: Group Secure Routes

```
Route::middleware(['auth']) ->group(function () {

    Route::get('/profile', function () {
        return "User Profile";
    });

    Route::get('/settings', function () {
        return "Settings Page";
    });

});
```

All routes inside the group are protected

STEP 1: Create Laravel Project (if not already)

composer create-project laravel/laravel myApp
cd myApp

STEP 2: Install Authentication (Breeze)

composer require laravel/breeze --dev
php artisan breeze:install
php artisan migrate
npm install && npm run dev

- This automatically sets up:
  a. Login / Register pages
  b. Auth controllers
  c. `auth` middleware

STEP 3: Run Server

php artisan serve

Open: http://127.0.0.1:8000

STEP 4: Add Secure Route

`routes/web.php`

```
use Illuminate\Support\Facades\Route;
Route::get('/', function () {
   return "Home Page";
});
//  ADD THIS ROUTE
Route::get('/dashboard', function () {
return "Welcome Dashboard";
})->middleware(['auth'])->name('dashboard')
```

STEP 5: Test Auth Middleware

Case 1: Not Logged In

Open: http://127.0.0.1:8000/dashboard

Output: Redirect to `/login`

Case 2: Logged In

Open: http://127.0.0.1:8000/register

- Create account
- Login
- Open `/dashboard`

Output:

This is Secure Dashboard

STEP 6: Using Controller

Create Controller

php artisan make:controller DashboardController

`app/Http/Controllers/DashboardController.php`

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class DashboardController extends Controller{
  public function __construct(){
    // Apply auth middleware
    $this->middleware('auth');
  }
  public function index(){
    return "Secure Dashboard from Controller";
  }
}
```

Add Route

`routes/web.php`

```
use App\Http\Controllers\DashboardController;
Route::get('/dashboard', [DashboardController::class, 'index']);
```

STEP 7: Route Group (Best Practice)

```
Route::middleware(['auth']) ->group(function () {
    Route::get('/profile', function () {
        return "Profile Page";
    });
    Route::get('/settings', function () {
        return "Settings Page";
    });
});
```

Domain Routing?

Domain routing allows you to handle **different domains or subdomains** in the same Laravel app.

example.com → Main site
admin.example.com → Admin panel
api.example.com → API routes

Basic Domain Routing

`routes/web.php`

```
Route::domain('admin.localhost') ->group(function () {
   Route::get('/', function () {
       return "Admin Panel";
   });
});
```

Access in Browser

http://admin.localhost:8000

You will see: **Admin Panel**

Important Setup : Add domain in hosts file

1. Windows: C:\Windows\System32\drivers\etc\hosts
2. Mac/Linux: /etc/hosts

Add: 127.0.0.1 admin.localhost

Multiple Domain Routing

```
Route::domain('admin.localhost') ->group(function () {
        Route::get('/dashboard', function () {
            return "Admin Dashboard";
        });
});

Route::domain('user.localhost') ->group(function () {
   Route::get('/dashboard', function () {
       return "User Dashboard";
   });
});
```

Dynamic Subdomain Routing

```
Route::domain('{account}.localhost') ->group(function () {
        Route::get('/', function ($account) {
            return "Welcome " . $account;
        });
});
```

Output

http://abc.localhost:8000 → Welcome abc

http://xyz.localhost:8000 → Welcome xyz

Using Controller (Optional)

```
Route::domain('admin.localhost') ->group(function () {
      Route::get('/home', [HomeController::class, 'home']);
});
```

Route Groups

Route groups allow you to **apply common properties** (middleware, prefix, namespace, etc.) to multiple routes.

Basic Syntax

```
Route::group([], function () {

        Route::get('/home', function () {
            return "Home";
        });

        Route::get('/about', function () {
            return "About";
        });
});
```

Example with Middleware

```
Route::middleware(['age.check']) ->group(function () {
    Route::get('/home', function () {
        return "Home Page";
        });
        Route::get('/dashboard', function () {
        return "Dashboard";
        });
});
```

Middleware applies to **all routes inside the group.**

Request → Group Middleware → Route → Response

Route Prefixing

Adds a **common URL path** before all routes in a group.

Basic Prefix Example

```
Route::prefix('admin')->group(function () {
    Route::get('/dashboard', function () {
            return "Admin Dashboard";
    });
    Route::get('/users', function () {
            return "Admin Users";
    });
});
```

Output URLs

/admin/dashboard

/admin/users

Combine Group + Prefix + Middleware

```
Route::prefix('admin')
   ->middleware(['age.check'])
   ->group(function () {
          Route::get('/dashboard', function () {
                 return "Admin Dashboard";
          });
          Route::get('/settings', function () {
                 return "Admin Settings";
          });
});
```

/admin → admin panel

/user → user panel

/api → API routes

Nested Groups (Advanced)

```
Route::prefix('admin')->group(function () {
        Route::prefix('users') ->group(function () {
        Route::get('/', function () {
                return "All Users";
        });
        Route::get('/create', function () {
                return "Create User";
        });
        });
});
```

/admin/users

/admin/users/create

```
Route::prefix('admin')->middleware(['age.check']) ->group(function () {
    Route::get('/dashboard', function () {
        return "Dashboard";
    });
});
```

URL Generation:

URL generation means creating **links (URLs)** to routes, controllers, or assets using Laravel helper functions instead of writing hardcoded URLs.

- URL generation in Laravel is the process of dynamically creating URLs using helper functions like `url()`, `route()`, and `asset()`.
- Avoid hardcoding URLs
- Easy maintenance
- It works even if route changes
- Cleaner code

1. Using `url()` Helper

```
$url = url('/home');
```

Output:

http://127.0.0.1:8000/home

Using `route()` Helper

Step 1: Define Named Route

```
Route::get('/home', function () {
        return "Home";
})->name('home');
```

Step 2: Generate URL

```
$url = route('home');
```

Output:

http://127.0.0.1:8000/home

Route with Parameters

```
Route::get('/user/{id}', function ($id) {
    return $id;
})->name('user');
```

Generate URL

```
$url = route('user', ['id' => 5]);
```

Output: /user/5

Using `asset()` (for CSS, JS, images)

```
echo asset('css/style.css');
```

Output: http://127.0.0.1:8000/css/style.css

Using `action()` (Controller URL)

```
use App\Http\Controllers\HomeController;
$url = action([HomeController::class, 'home']);
```

In Blade

```
<a href="{{ url('/home') }}">Home</a>
<a href="{{ route('home') }}">Home</a>
<a href="{{ route('user', 5) }}">User</a>
<link rel="stylesheet" href="{{ asset('css/style.css') }}"
```

1. Current URL:

- To retrieve and display the current URL, you can do the following:
- Open an existing Blade view file or create a new one (e.g., resources/views/current-url.blade.php).
- In your Blade view file, add the following code to display the current URL.
- In current.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Current URL Example</title>
</head>
<body>
    <p>Current URL: {{ url()->current() }}</p>
    <p>Current URL: {{ url()->previous() }}</p>
    <p>Current URL: {{ url()->full() }}</p>
</body>
</html>
```

In web.php and in UserController

In web.php

```
// Route to display the current URL example
Route::get('/current-url', [UserController::class, 'currentUrl’]);
```

UserController function

```
public function currentUrl()
    {
        return view('current');
    }
```

Browser Output

Current URL: http://localhost:8000/current-url

2. Generating Framework URLs:

- To generate framework URLs for named routes and controller actions, follow these steps:
- Open an existing Blade view file or create a new one (e.g., resources/views/framework-urls.blade.php).
- In your Blade view file, you can generate URLs using the route() and action() functions. For example:

```
<!DOCTYPE html>
<html>
<head>
    <title>Framework URLs Example</title>
</head>
<body>
    @php
    use App\Http\Controllers\UserController;
@endphp

    <p>Profile URL: <a href="{{ route('profile') }}">Profile</a></p>
    <p>User Edit URL: <a href="{{ action([UserController::class,
'showProfile']) }}">My Profile</a></p>
</body>
</html>
```

3. Asset URLs:

To generate URLs for asset files (CSS, JavaScript, images), follow these steps:

```
<!DOCTYPE html>
<html>
<head>
    <title>Asset URLs Example</title>
    <link rel="stylesheet" href="{{ asset('css/style.css') }}">
</head>
<body>
    <!-- Your HTML content here -->
</body>
</html>
```

4. Generation Shortcuts:

Laravel provides shortcuts for generating URLs directly in Blade templates. You can use these shortcuts within your Blade views. For example:

```
<!DOCTYPE html>
<html>
<head>
    <title>Shortcuts Example</title>
</head>
<body>
    <p>About Us: <a href="{{ url('/admin/dashboard')
}}">AboutUs</a></p>
    <p>Profile: <a href="{{ route('profile') }}">Profile</a></p>
</body>
</html>
```